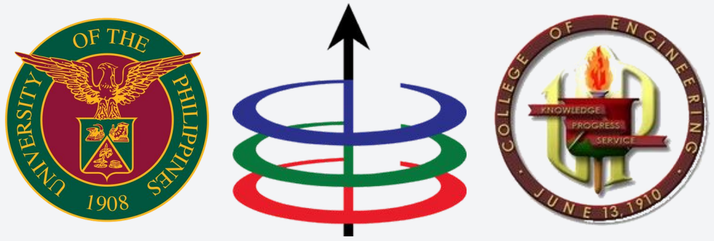


Lecture 6
Graph Representation and Traversal

EEE 121 2s2526

Steven S. Sison

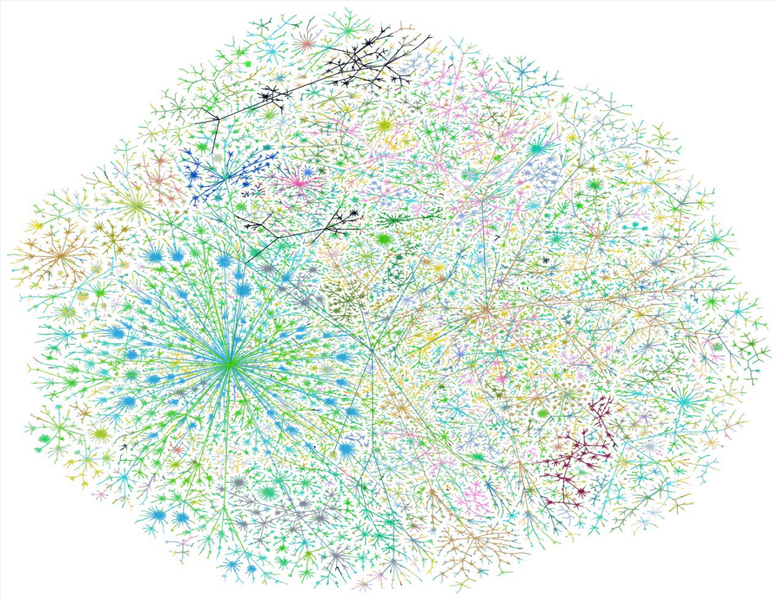
- Graph Representation
- Graph Traversal
- Directed Acyclic Graphs

- **Graph Representation**
- Graph Traversal
- Directed Acyclic Graphs

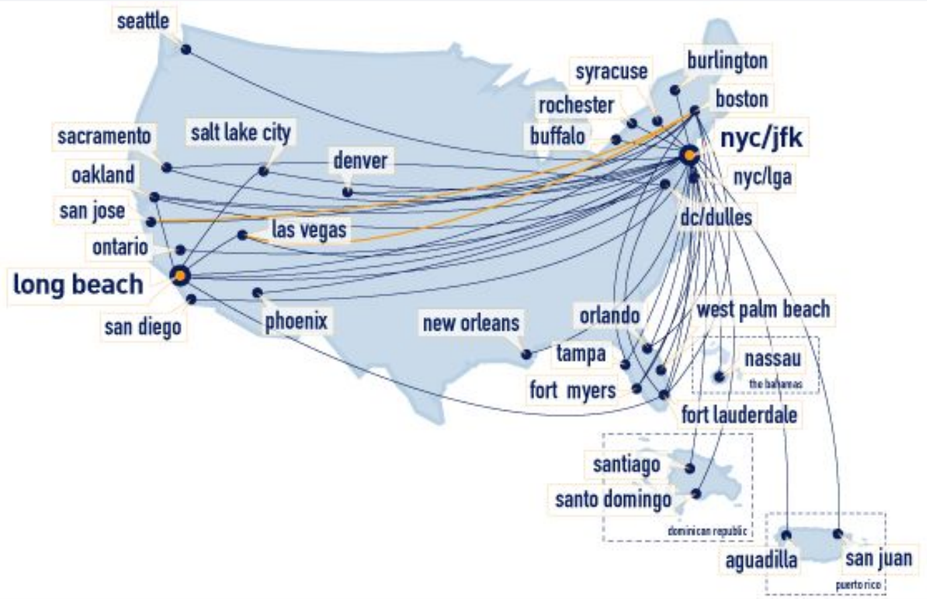
Graph Fundamentals

A **graph** is a way of representing relationships that exists between pairs of objects.

- Vertices - these are the set of objects in the graph
- Edges - these are the set of connections between the objects in the graph



The Internet as a graph



Flight Network

Undirected Graph vs Directed Graph

Formally, a graph is represented by $G(V,E)$.

UNDIRECTED GRAPH

- V is the set of vertices
- E is the set of edges

Example:

- V = {A,B,C,D}
- E = {(A,B), (A,C),(A,D),(B,C),(C,D)}

DIRECTED GRAPH

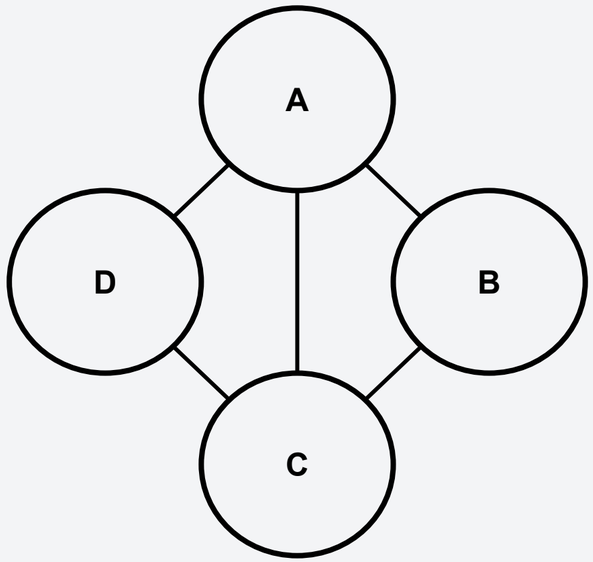
- V is the set of vertices
- E is the set of **directed** edges

Example:

- V = {A,B,C,D}
- E = {(A,B), (A,D),(B,C),(C,A),(D,A),(D,C)}

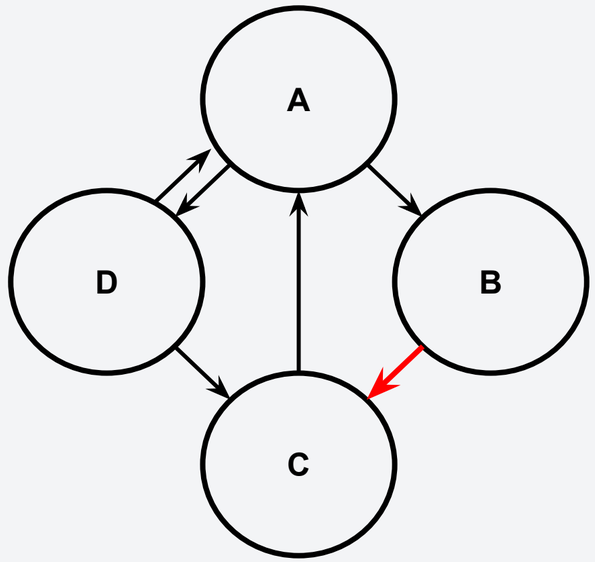
Undirected Graph vs Directed Graph

UNDIRECTED GRAPH



- V = {A,B,C,D}
- E = {(A,B), (A,C),(A,D),(B,C),(C,D)}

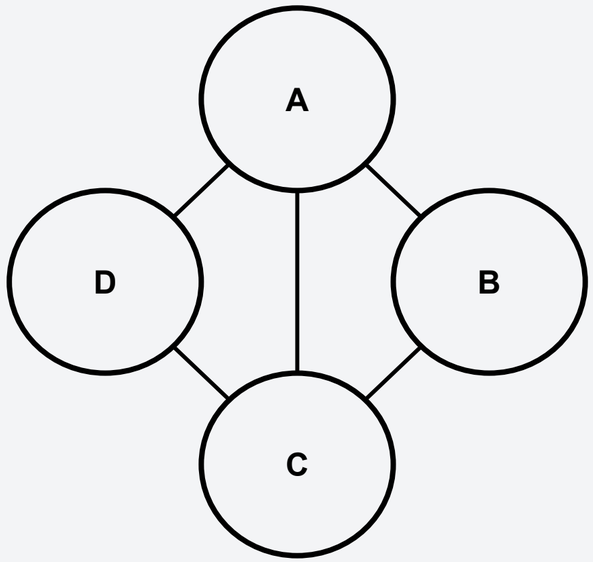
DIRECTED GRAPH



- V = {A,B,C,D}
- E = {(A,B), (A,D),(B,C),(C,A),(D,A),(D,C)}

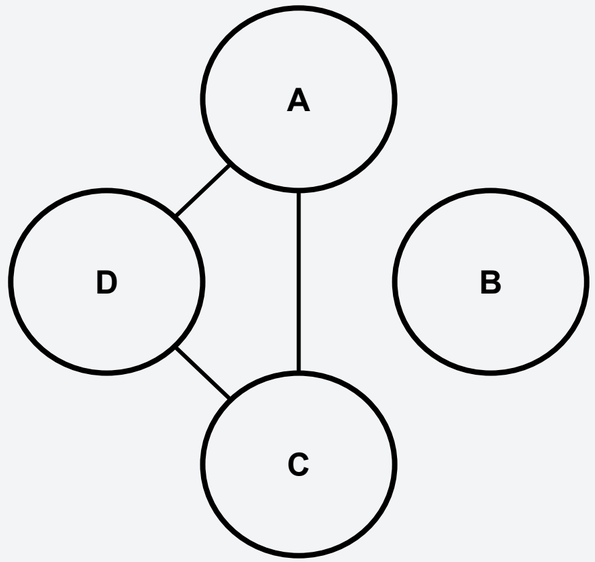
Connected vs Disconnected Graph

CONNECTED GRAPH



- All nodes have edges.

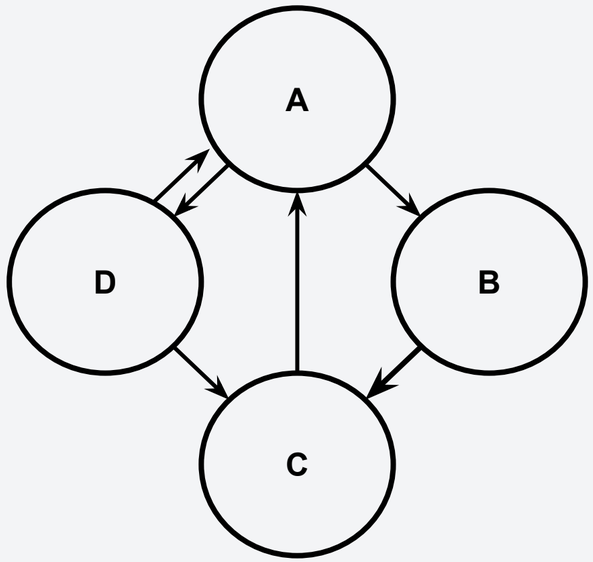
DISCONNECTED GRAPH



- Some nodes have no edges and are therefore, isolated.

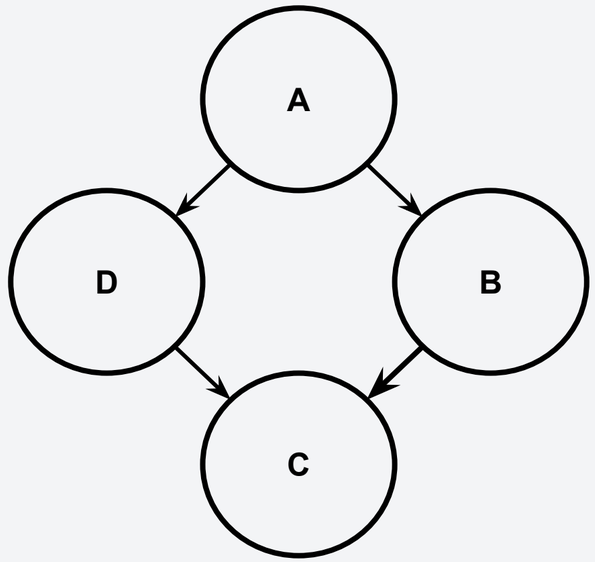
Cyclic vs Acyclic Graph

CYCLIC GRAPH



- The graph contains a loop.
- Ex. A -> D -> C -> A

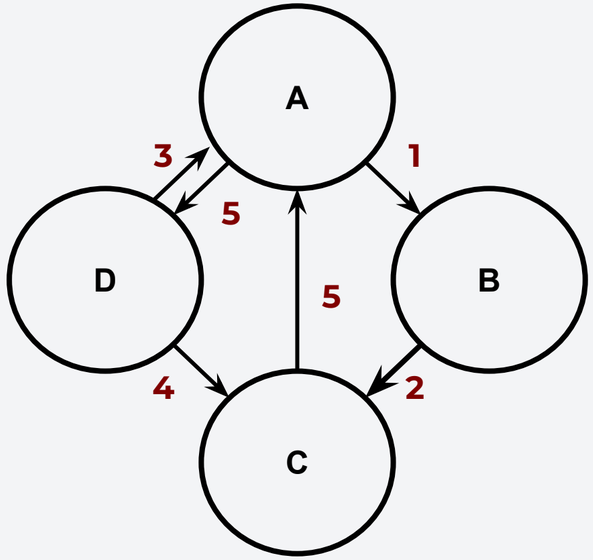
ACYCLIC GRAPH



- The graph contains no loop.

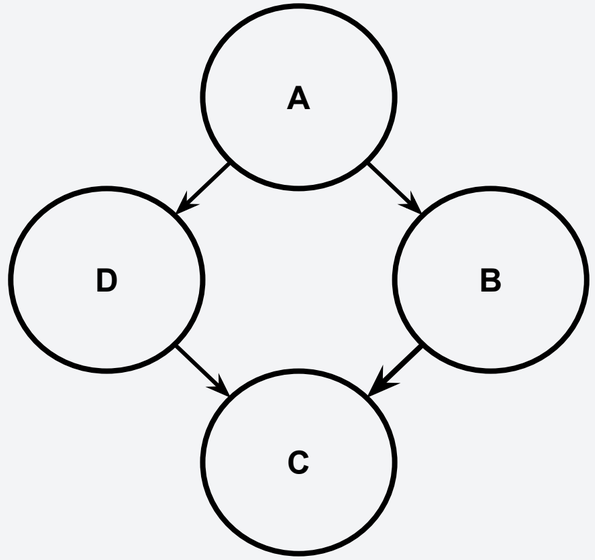
Weighted vs Unweighted Graph

WEIGHTED GRAPH



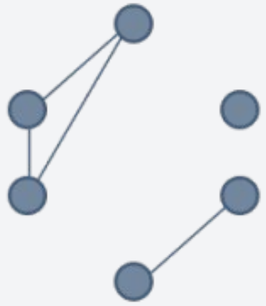
- Each edge has a corresponding weight.

UNWEIGHTED GRAPH

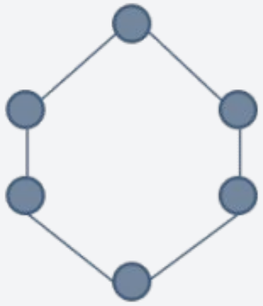


- Edges have no specified weight.

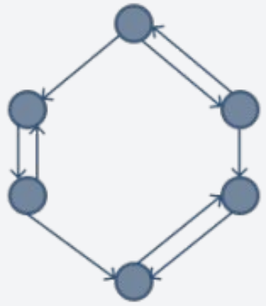
Types of Graphs - Summary



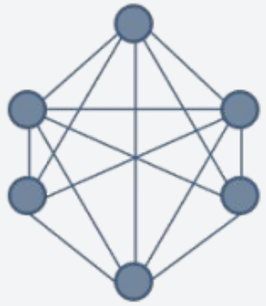
disconnected



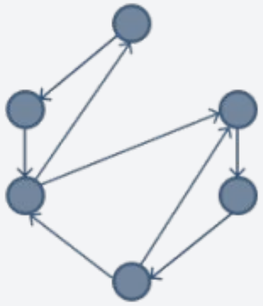
connected & undirected



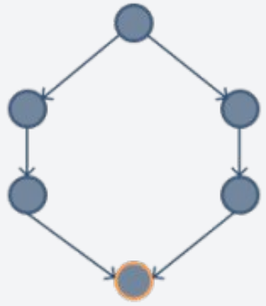
directed



complete



cyclic



acyclic

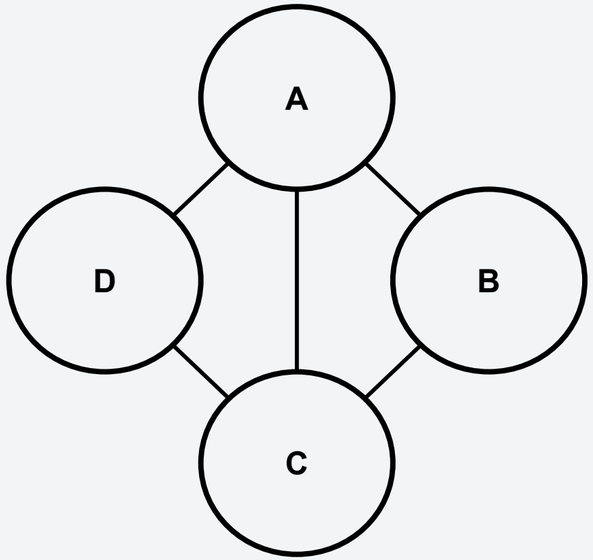
Graph Terminologies

- The **degree** of a vertex is defined as the number of of adjacent vertices.
- Vertices that are adjacent to a given vertex are considered as **neighbors**.
- A **sub-graph** of a graph is a subset of the vertices and a subset of the edges that connects the subset of the vertices in the original graph.

Graph Terminologies

- The **degree** of a vertex is defined as the number of of adjacent vertices.
- Vertices that are adjacent to a given vertex are considered as **neighbors**.
- A **sub-graph** of a graph is a subset of the vertices and a subset of the edges that connects the subset of the vertices in the original graph.

What are the degree of vertex A? B? C? D?

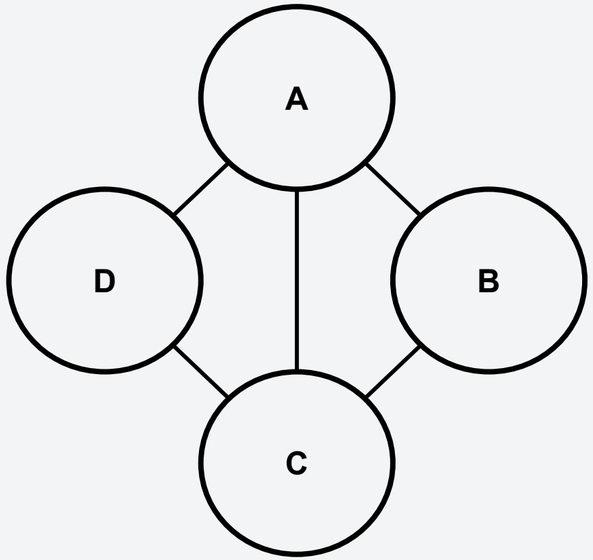


Graph Terminologies

- The **degree** of a vertex is defined as the number of of adjacent vertices.
- Vertices that are adjacent to a given vertex are considered as **neighbors**.
- A **sub-graph** of a graph is a subset of the vertices and a subset of the edges that connects the subset of the vertices in the original graph.

What are the degree of vertex A? B? C? D?

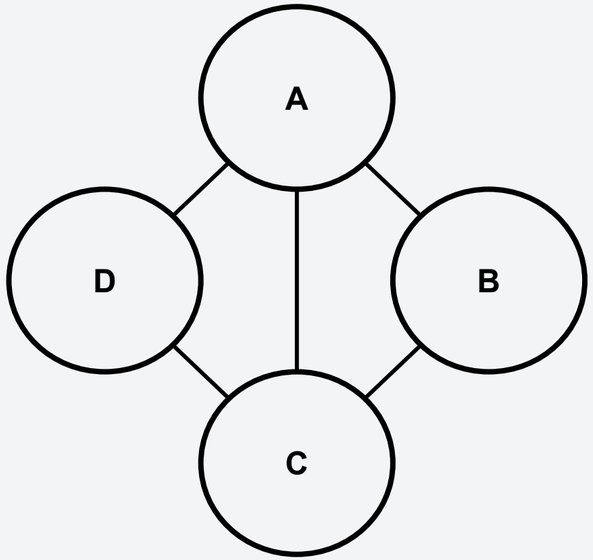
- **deg(A) = deg(C) = 3**
- **deg(B) = deg(D) = 2**



Graph Terminologies

- The **degree** of a vertex is defined as the number of of adjacent vertices.
- Vertices that are adjacent to a given vertex are considered as **neighbors**.
- A **sub-graph** of a graph is a subset of the vertices and a subset of the edges that connects the subset of the vertices in the original graph.

Is B a neighbor of D? If yes, from what vertex?

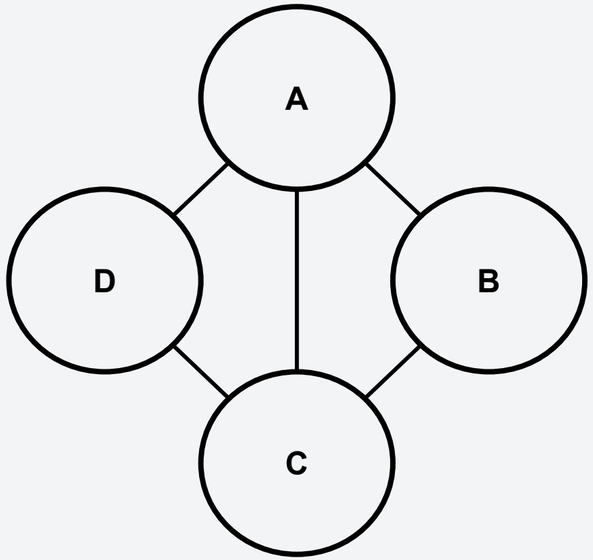


Graph Terminologies

- The **degree** of a vertex is defined as the number of of adjacent vertices.
- Vertices that are adjacent to a given vertex are considered as **neighbors**.
- A **sub-graph** of a graph is a subset of the vertices and a subset of the edges that connects the subset of the vertices in the original graph.

Is B a neighbor of D? If yes, from what vertex?

- **Yes. From vertex A.**

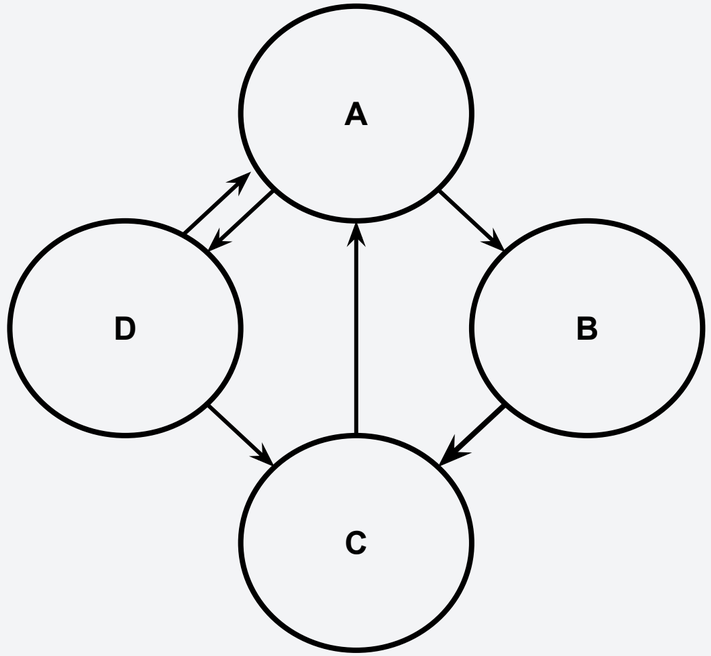


Graph Terminologies

For directed graphs, the edge direction matters in the degree.

- **Indegree**: the total of incoming edges to a vertex
- **Outdegree**: the total of outgoing edges to a vertex

What are the indegree and outdegree of vertex A?



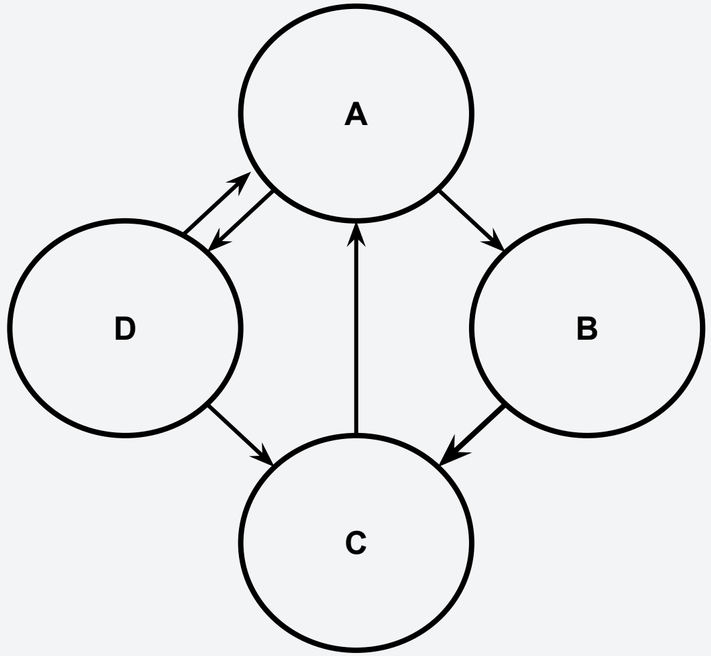
Graph Terminologies

For directed graphs, the edge direction matters in the degree.

- **Indegree**: the total of incoming edges to a vertex
- **Outdegree**: the total of outgoing edges to a vertex

What are the indegree and outdegree of vertex A?

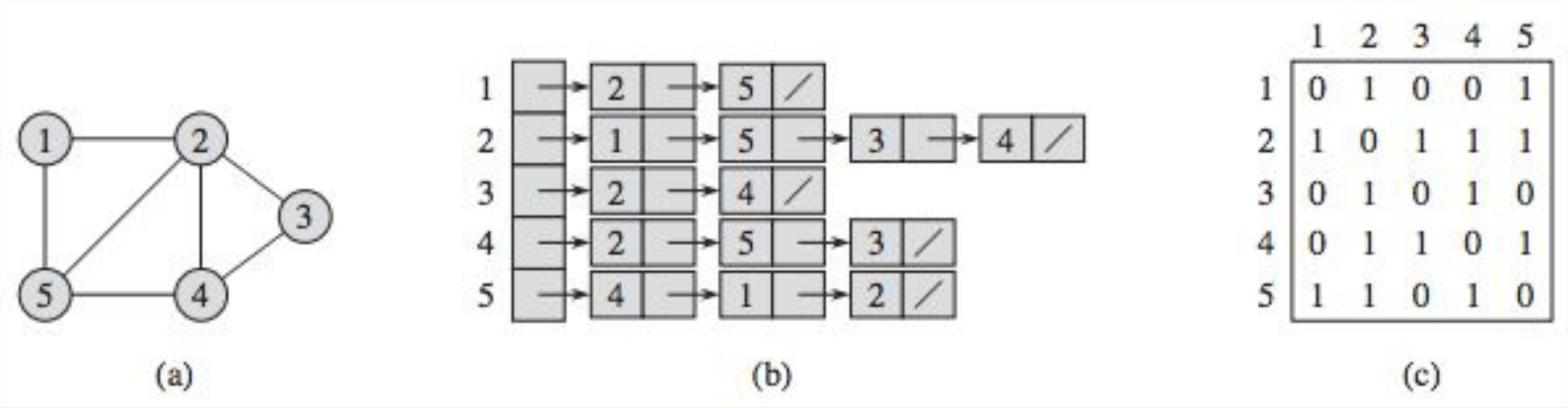
- **Indegree: 2**
- **Outdegree: 2**



Representing Graphs

There are two ways that we can represented a graph:

1. Adjacency Matrix (shown in c)
2. Adjacency List (shown in b)



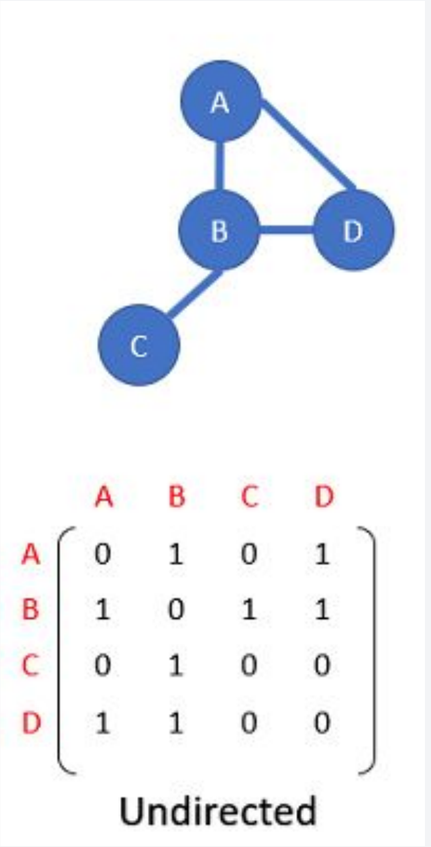
Representing Graphs – Adjacency Matrix

An adjacency matrix is essentially a two-dimensional structure that contains a non-zero value, usually the weight, if there is a connection between the edges.

For an undirected graph,

- Each vertices are assigned an index
- For an undirected graph, we set a[i][j] to 1 if there is a connection between vertex i and j.

The resulting matrix is symmetric along the diagonal



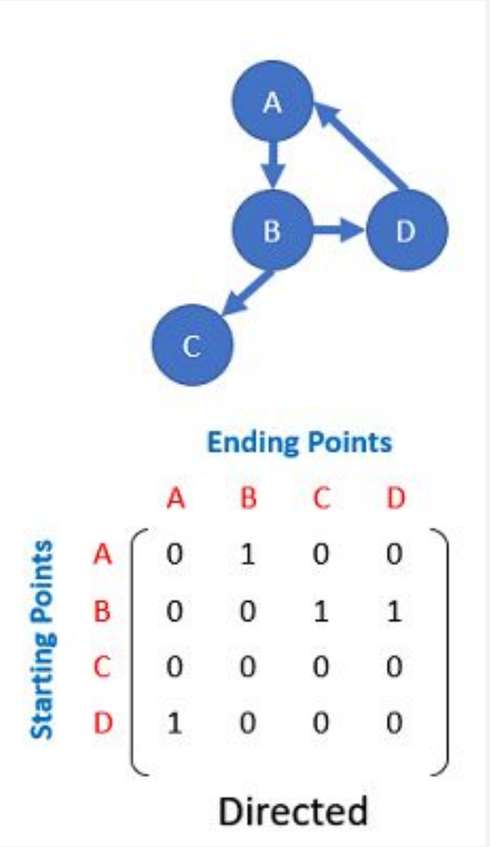
Representing Graphs – Adjacency Matrix

An adjacency matrix is essentially a two-dimensional structure that contains a non-zero value, usually the weight, if there is a connection between the edges.

For a directed graph,

- Each vertices are assigned an index
- For a directed graph, we set a[i][j] to 1 if there is a connection from vertex i to vertex j. However, this does not indicate any connection from vertex j to vertex i.

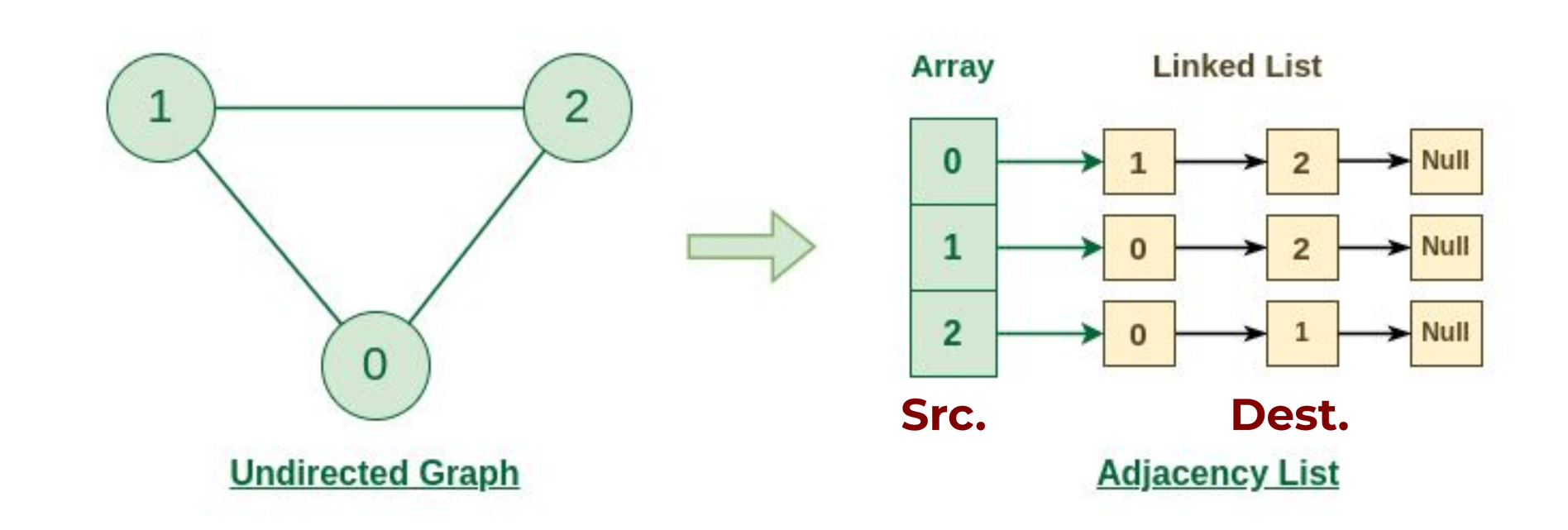
Matrix is no longer symmetric



Representing Graphs – Adjacency List

On the other hand, adjacency list represent graphs by using list wherein its indices correspond to the vertices of the graph.

- Each vertex is given an index. If vertex A is index 0, its connections will be stored by the list at index 0.



Representing Graphs – Comparison

To compare the graphs, we look at their time and space complexities in common operations done in the graph:

For graph, G(V,E)	Adjacency Matrix	Adjacency List
Edge Membership Is e={u,v} in E?	$O(1)$	$O(deg(u))$ or $O(deg(v))$
Neighbor Query What are the neighbors of u?	$O(\|V\|)$	$O(deg(u))$
Space Requirement	$O(\|V\|^2)$	$O(\|V\|+\|E\|)$

The Adjacency List is generally better for sparse graphs.

Sparse = more zeros than 1 = less connected edges

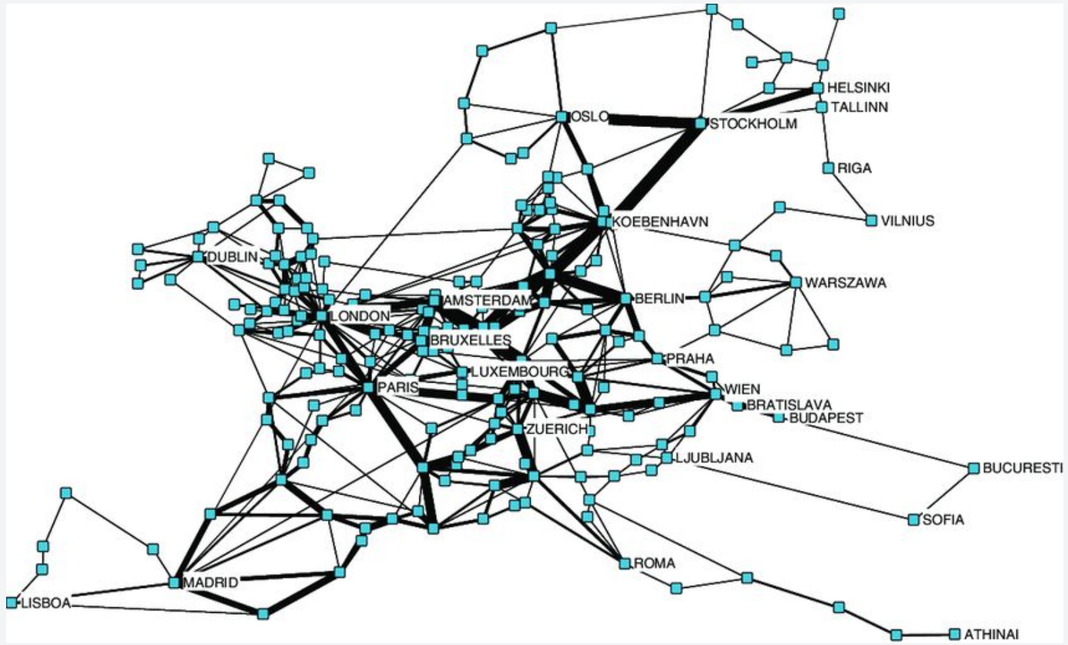
- Graph Representation
- **Graph Traversal**
- Directed Acyclic Graphs

Graph kayo ng graph, ano ba silbe ng graph na yan?

A lot of real-life problems are solvable by using graph algorithms. For example,

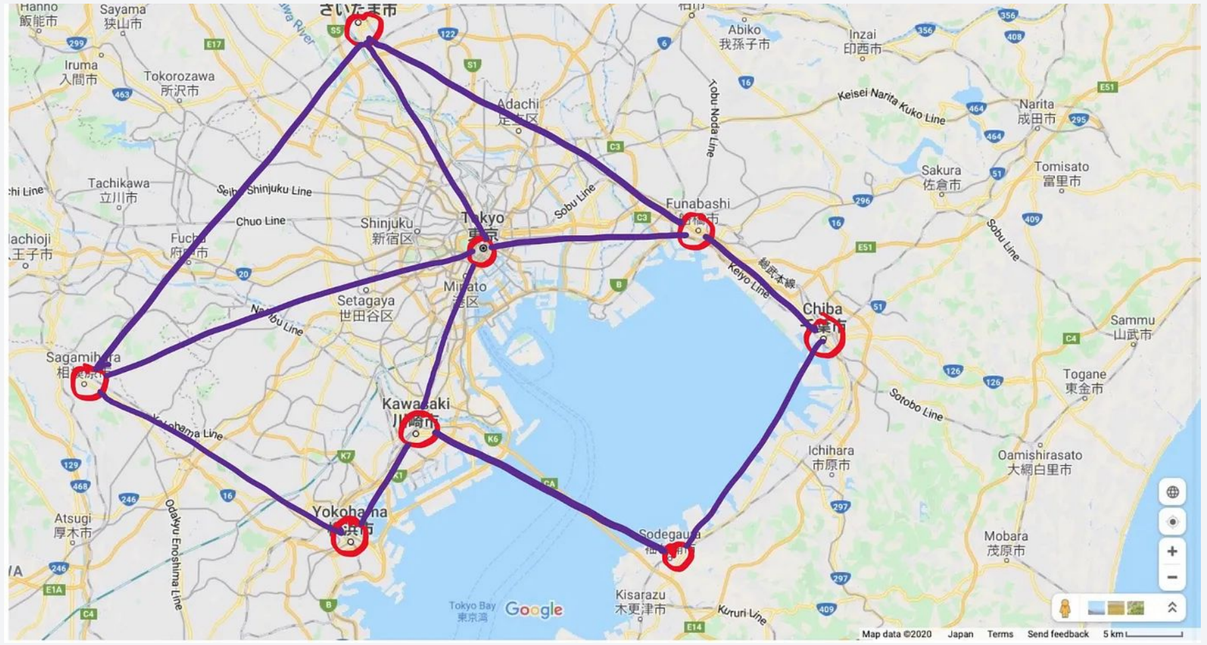
1. In a computer network, you want to make sure that all computers are connected.
    a. Represent the network as a graph and traverse the graph. This will verify if a certain node is reachable by all the nodes of the network.
2. In a road network, you want to close a certain road for maintenance purposes. You want to ensure that all areas are still accessible.
    a. Similar concept to item 1.
3. In a navigation system, you want to return the shortest path possible.
    a. Represent the road network as a graph with a weighted edge representing the travel time. Traverse the graph to find the shortest path possible using Dijkstra’s algorithm.

Graph kayo ng graph, ano ba silbe ng graph na yan?



Internet as a graph

Determine if all nodes are reachable through graph traversal



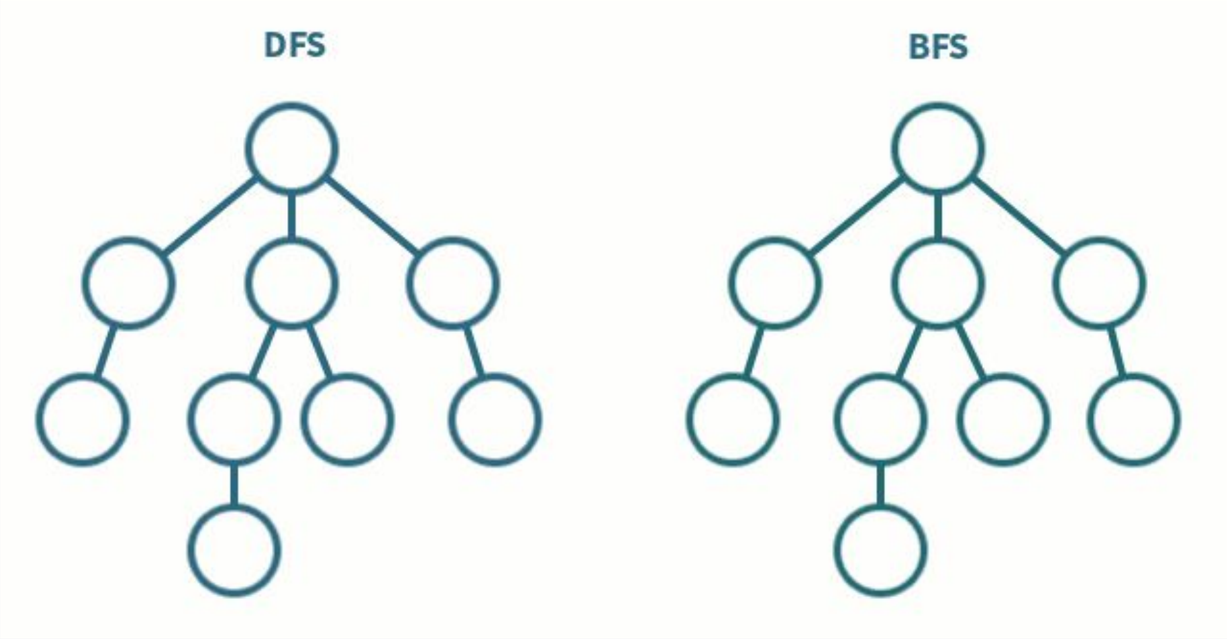
Road Networks as a graph

Find the shortest path possible between two nodes by traversing all possible paths

Traversing Graphs

Similar to trees, there are different ways that we can traverse graphs. The most common algorithm include:

1. Depth-first search (DFS)
2. Breadth-first search (DFS)



Depth-first Search (DFS)

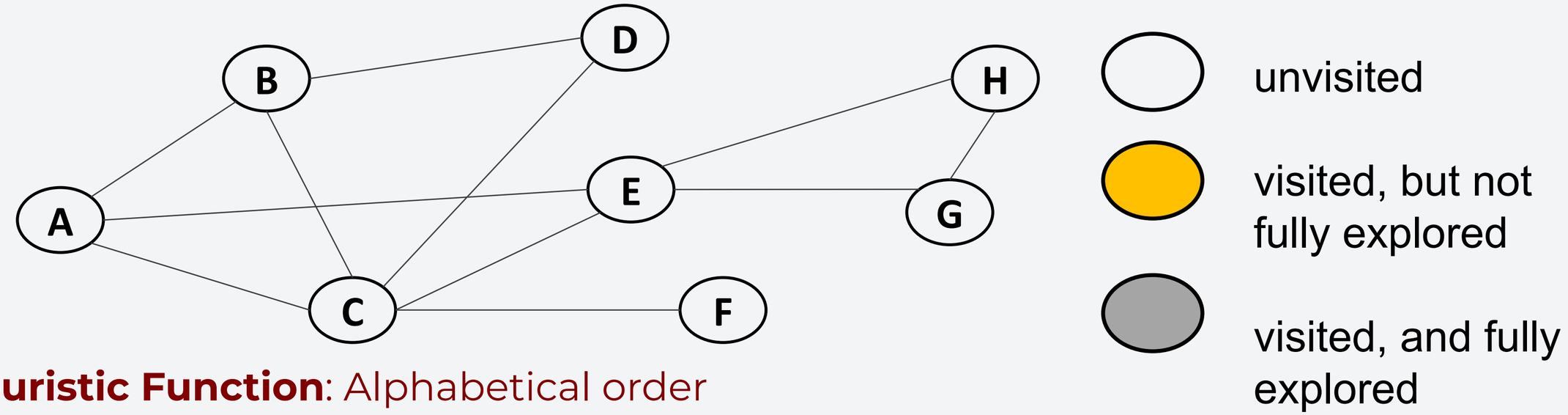
Traverse the graph by exploring the deepest nodes first then working upwards. To do DFS, perform the following steps:

1. Choose any vertex. Mark it as visited.
2. From that vertex:
    a. If there is another adjacent vertex not yet visited, go to it.
    b. Otherwise, go back to the previous vertex that has not yet had all of its adjacent vertices visited and continue from there.
    c. Continue until no visited vertices have unvisited adjacent vertices.

Key terms: visited - nodes that were visited but still have unvisited vertices
explored - nodes that were visited and no longer have unvisited vertices

DFS Example

Given the following graph, let's do a DFS:



Heuristic Function: Alphabetical order

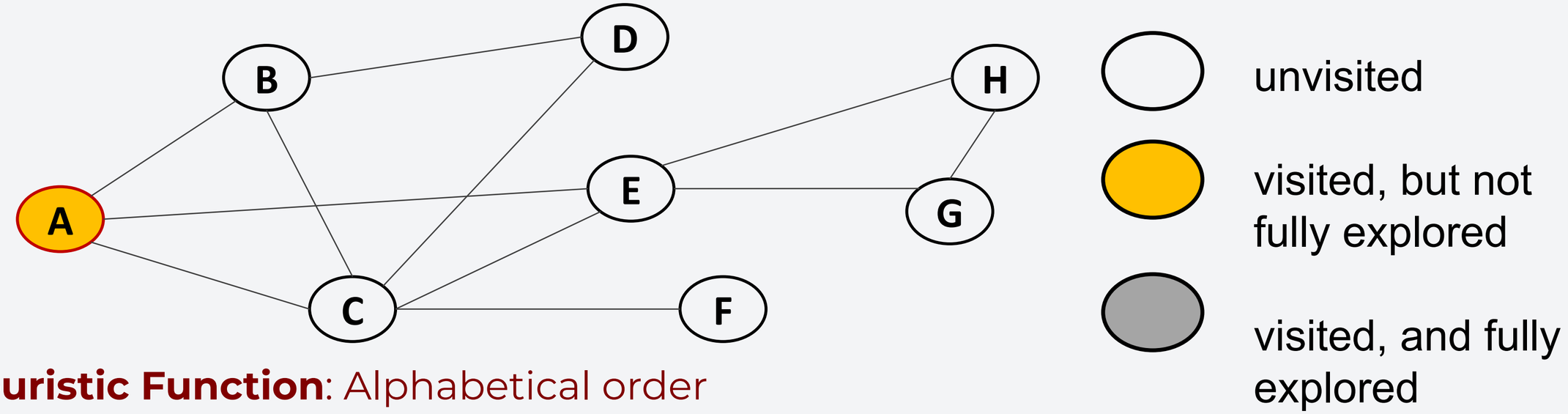
Visited:

Explored:

DFS Example

Given the following graph, let's do a DFS:

- Select vertex A as the starting vertex. Mark it as visited but not fully explored.



Heuristic Function: Alphabetical order

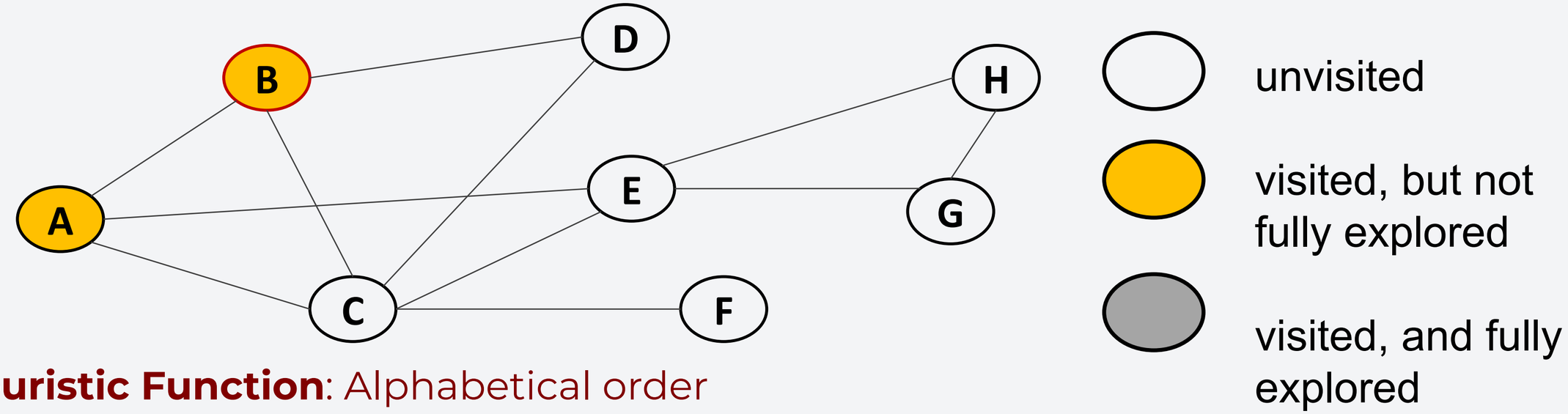
Visited: A

Explored:

DFS Example

Given the following graph, let's do a DFS:

- Since A has unvisited vertices, we choose and visit one.
- Choose B, mark it as visited but not fully explored since it still has unvisited vertices.



Heuristic Function: Alphabetical order

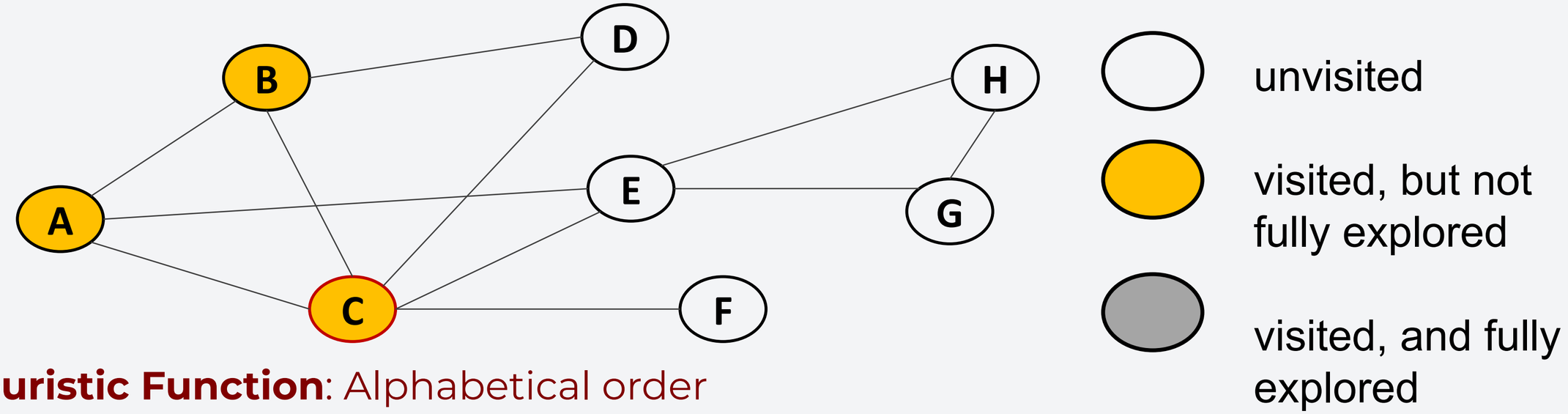
Visited: A, B

Explored:

DFS Example

Given the following graph, let's do a DFS:

- B has vertices C and D as unvisited vertices, choose C.
- Visit C and mark is as visited, but not fully explored.



Heuristic Function: Alphabetical order

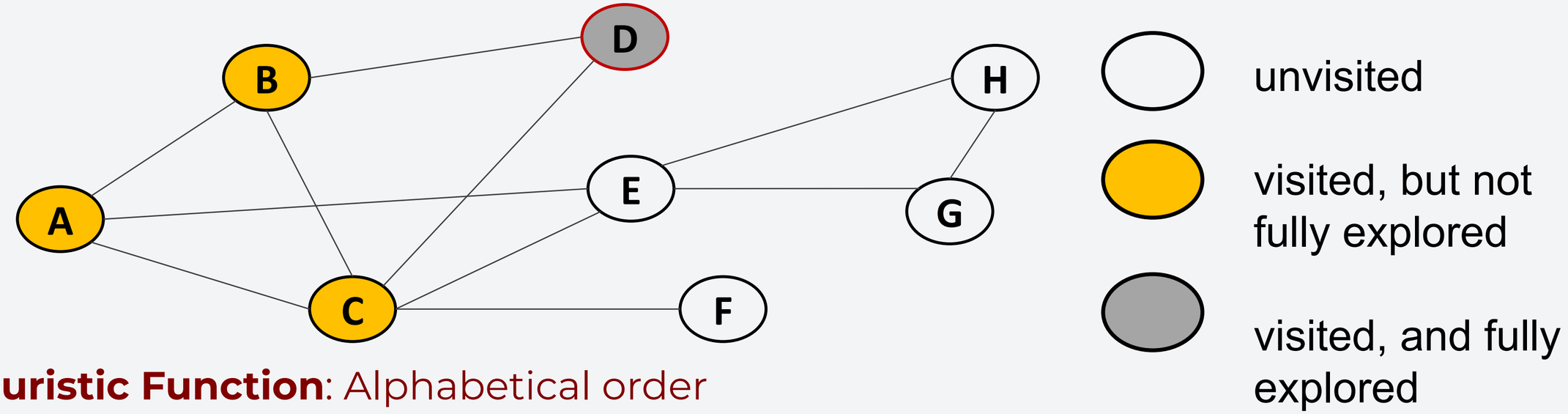
Visited: A, B, C

Explored:

DFS Example

Given the following graph, let's do a DFS:

- C has unvisited vertices (D, E, and F). Visit D.
- Mark D as fully explored since it no longer has unvisited vertices.



Heuristic Function: Alphabetical order

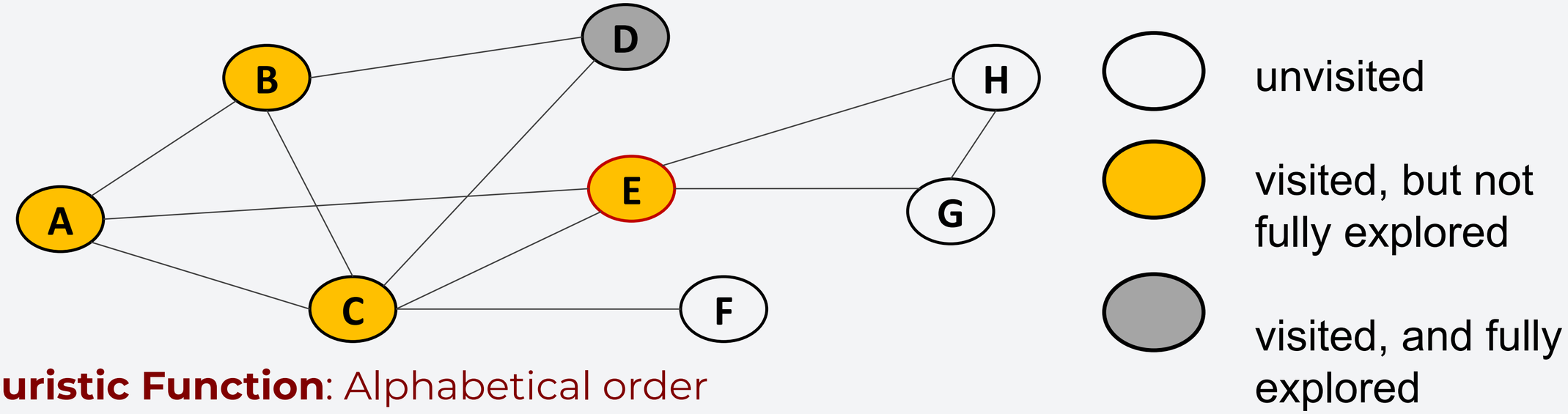
Visited: A, B, C, D

Explored: D

DFS Example

Given the following graph, let's do a DFS:

- D has no unvisited vertices. Go back to C.
- Visit E, mark it as visited, but not fully explored.



Heuristic Function: Alphabetical order

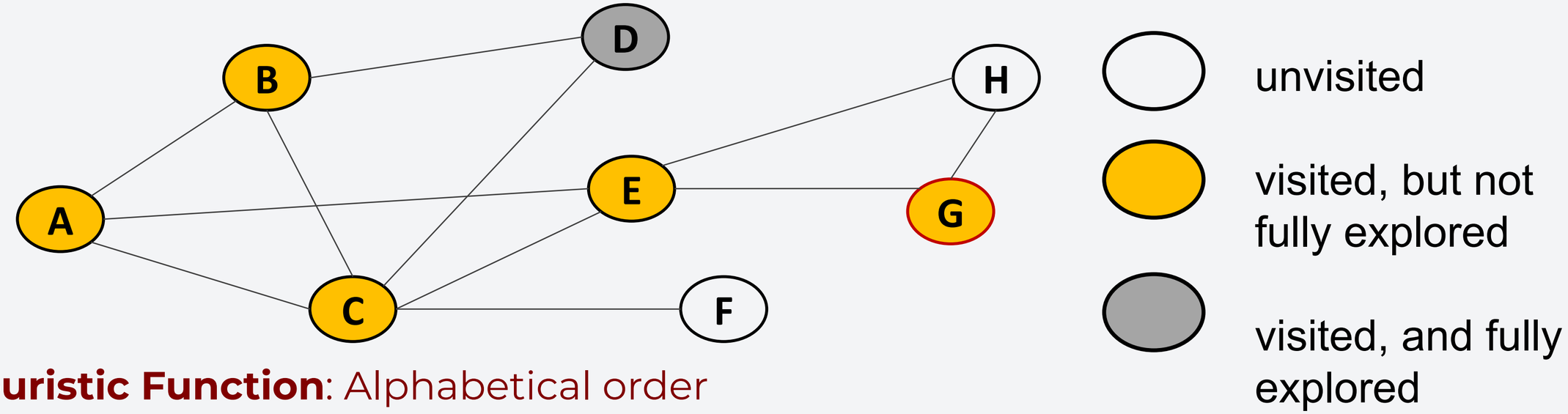
Visited: A, B, C, D, E

Explored: D

DFS Example

Given the following graph, let's do a DFS:

- E has unvisited vertices (H and G).
- Visit G and mark it as visited, but not fully explored.



Heuristic Function: Alphabetical order

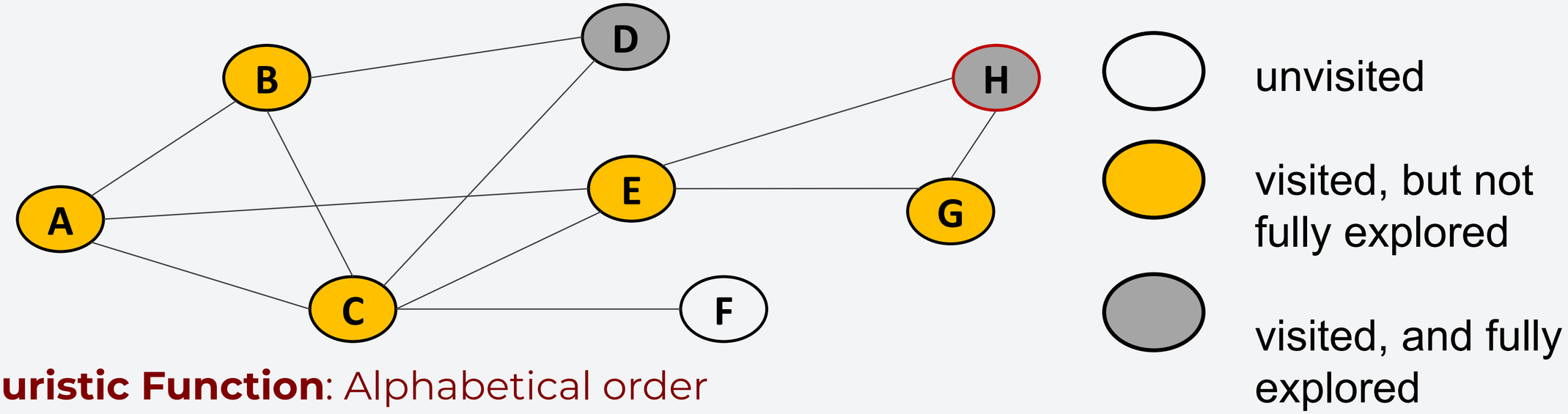
Visited: A, B, C, D, E, G

Explored: D

DFS Example

Given the following graph, let's do a DFS:

- G has an unexplored vertex (H).
- Visit H and mark it as fully explored since it no longer has unvisited vertices.



Heuristic Function: Alphabetical order

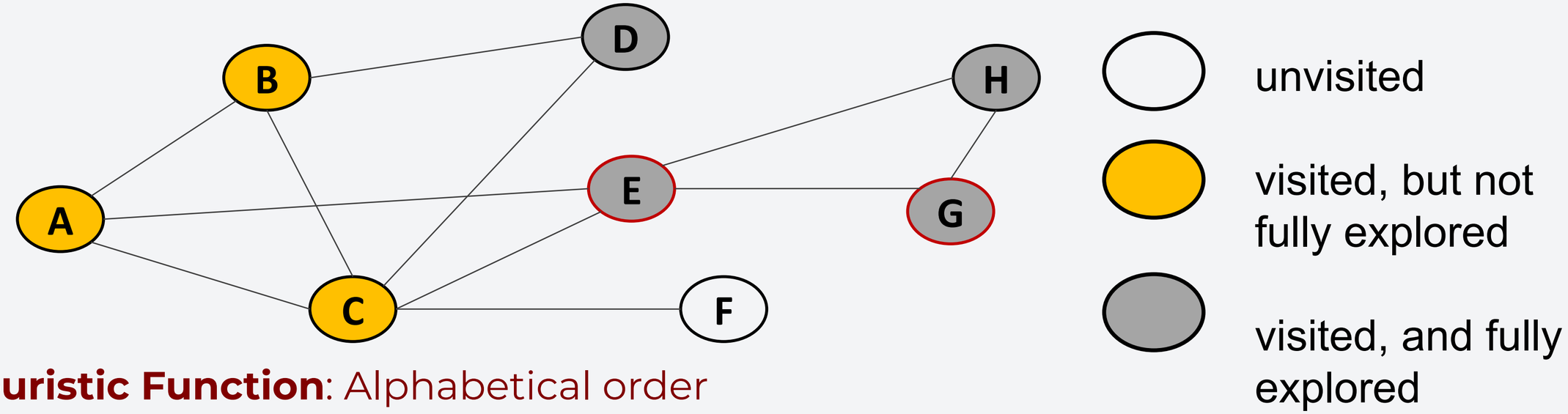
Visited: A, B, C, D, E, G, H

Explored: D, H

DFS Example

Given the following graph, let's do a DFS:

- Go back to G and mark it as fully explored.
- Go back to E and mark it as fully explored.



Heuristic Function: Alphabetical order

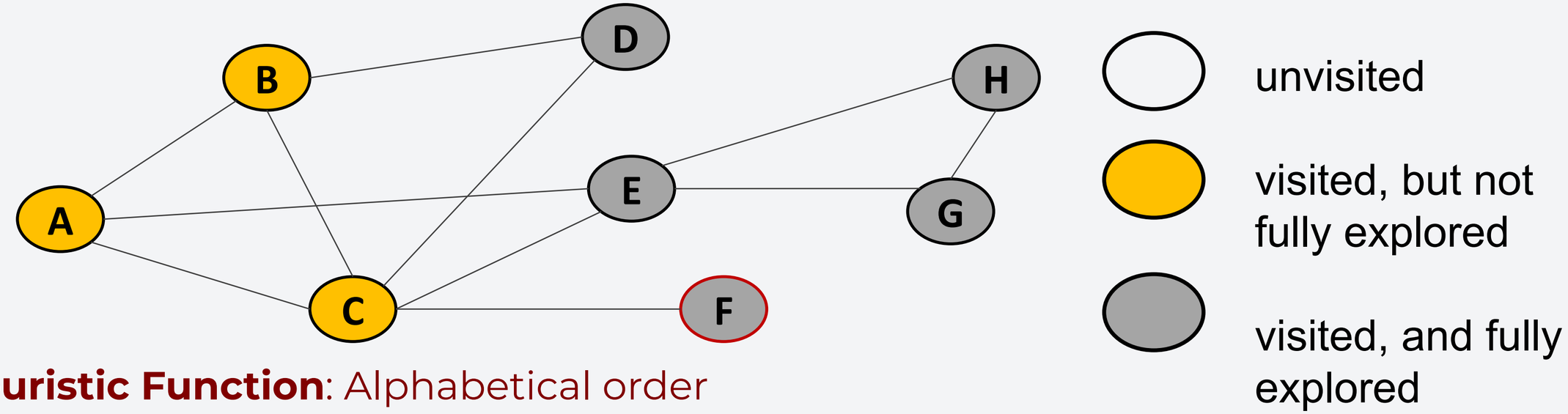
Visited: A, B, C, D, E, G, H

Explored: D, H, G, E

DFS Example

Given the following graph, let's do a DFS:

- Finally go back to C and visit F.
- Since F no longer has unvisited vertices, mark it as fully explored.



Heuristic Function: Alphabetical order

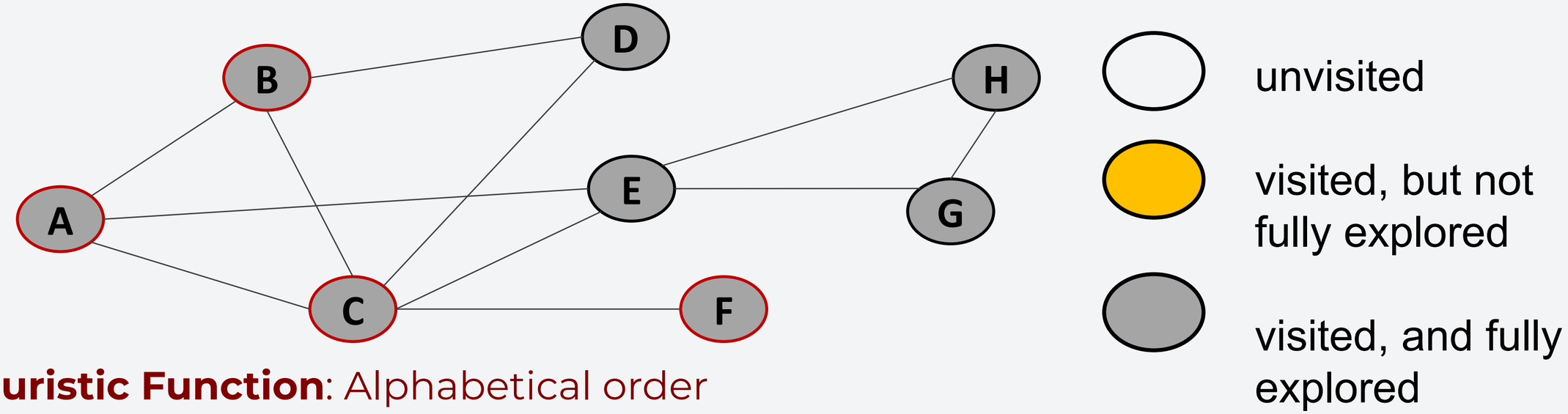
Visited: A, B, C, D, E, G, H, F

Explored: D, H, G, E, F

DFS Example

Given the following graph, let's do a DFS:

- Go back to C, mark it as fully explored. Go back to B, mark as fully explored as well.
- Go back A and mark it as fully explored.



Heuristic Function: Alphabetical order

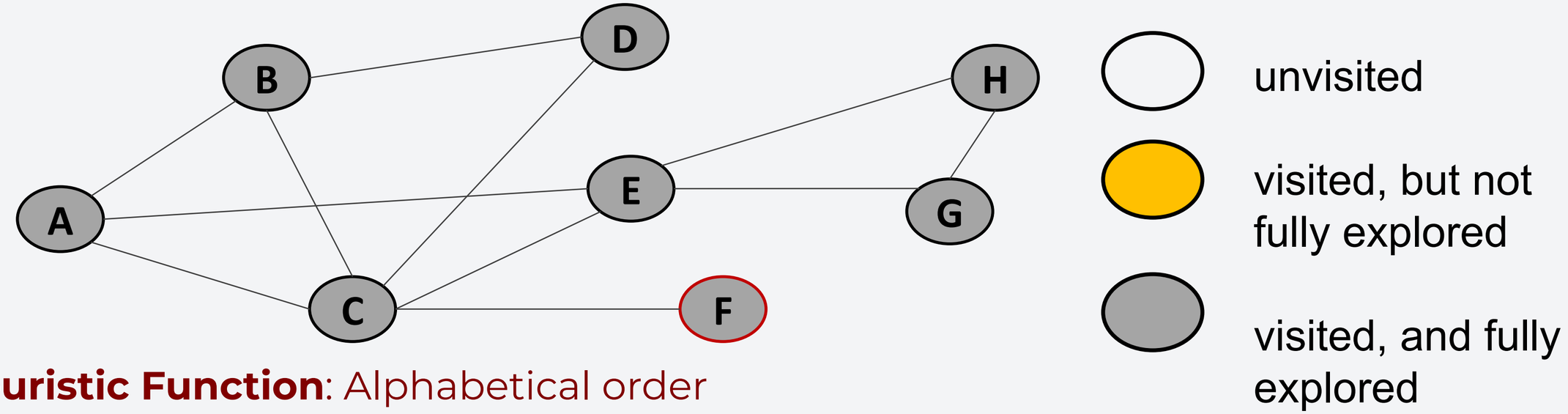
Visited: A, B, C, D, E, G, H, F

Explored: D, H, G, E, F, C, B, A

DFS Example

Given the following graph, let's do a DFS:

- All nodes are now traversed.



Heuristic Function: Alphabetical order

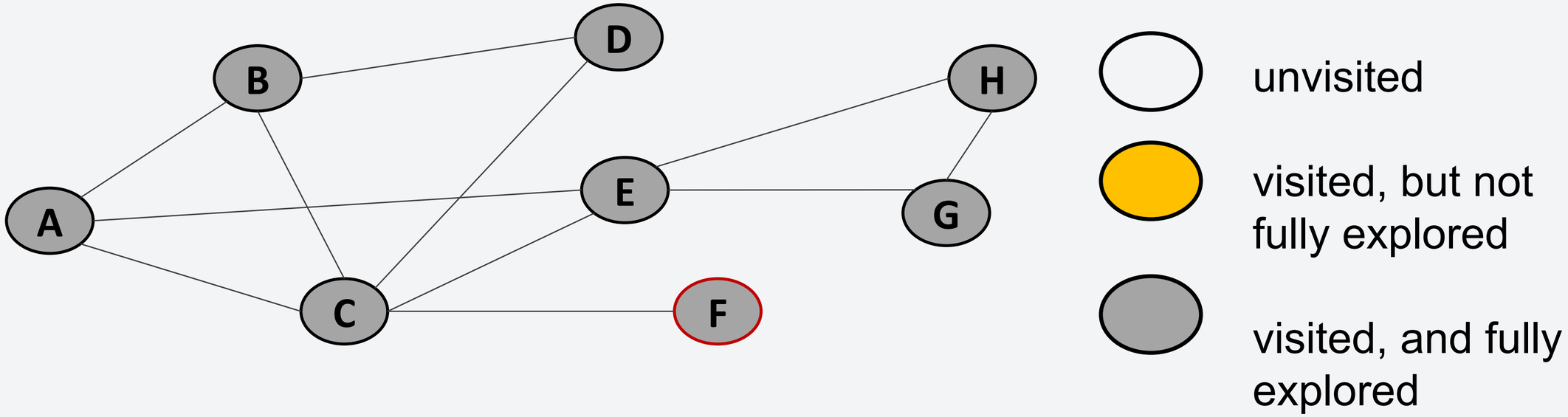
Visited: A, B, C, D, E, G, H, F

Explored: D, H, G, E, F, C, B, A

DFS Example

Given the following graph, let's do a DFS:

- All nodes are now traversed.



DFS can be implemented recursively or iteratively. You will implement this in activity 6 :D

Demo: Depth-First Search

Breadth-first Search (DFS)

Traverse the graph by nodes of the same level first then work downwards per level. To do BFS, perform the following steps:

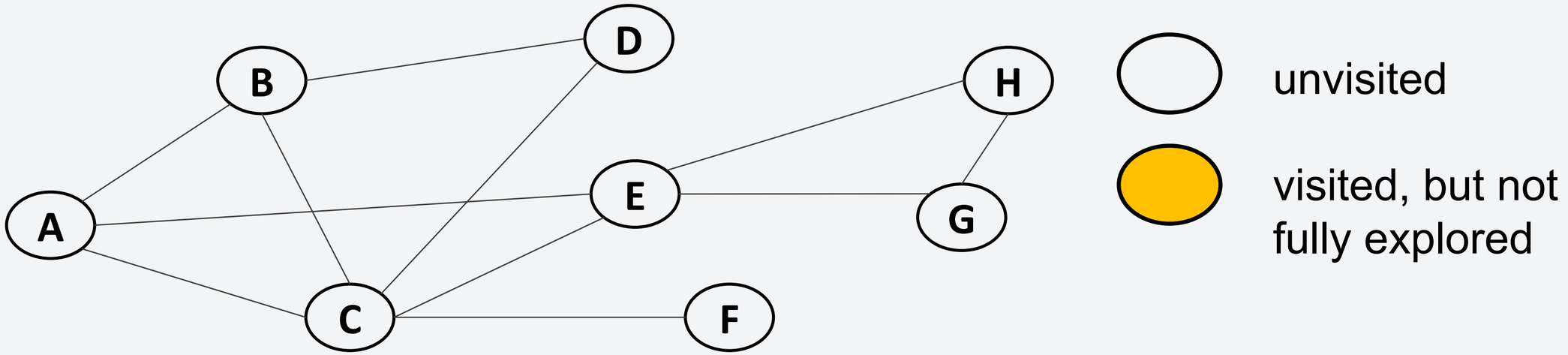
1. Choose any vertex, mark it as visited, and enqueue it onto a queue.
2. While the queue is not empty:
    a. Dequeue vertex v from the queue.
    b. For each unvisited neighbors of v:
        i. Mark it visited and enqueue it.
3. Perform the while loop until all nodes are visited.

Note. If there is no node unvisited, the graph is connected.

Note. If we change the data structure to stack, we can implement DFS.

BFS Example

Given the following graph, let's do a BFS:



Heuristic Function: Alphabetical order

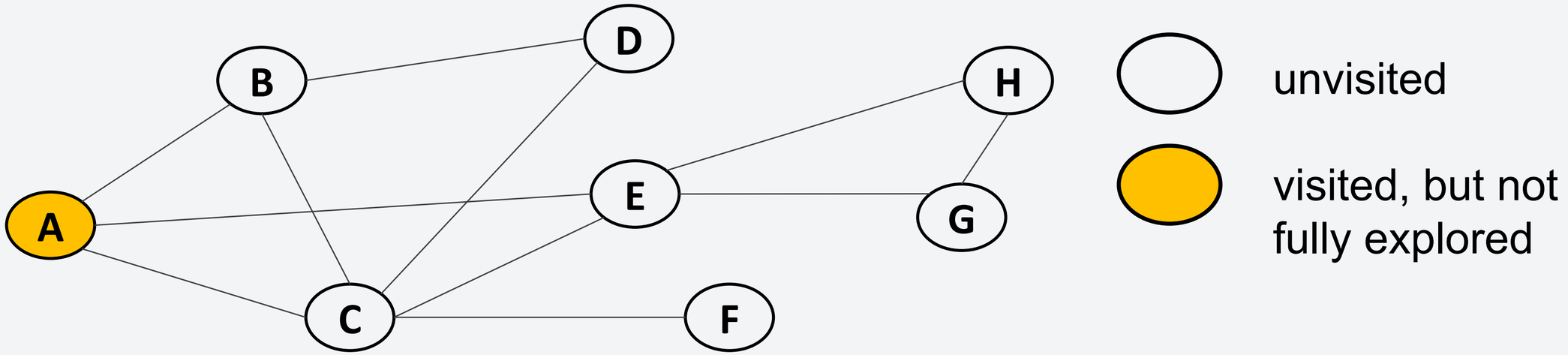
Queue:

Visited:

BFS Example

Given the following graph, let's do a BFS:

- Visit A and enqueue it.



Heuristic Function: Alphabetical order

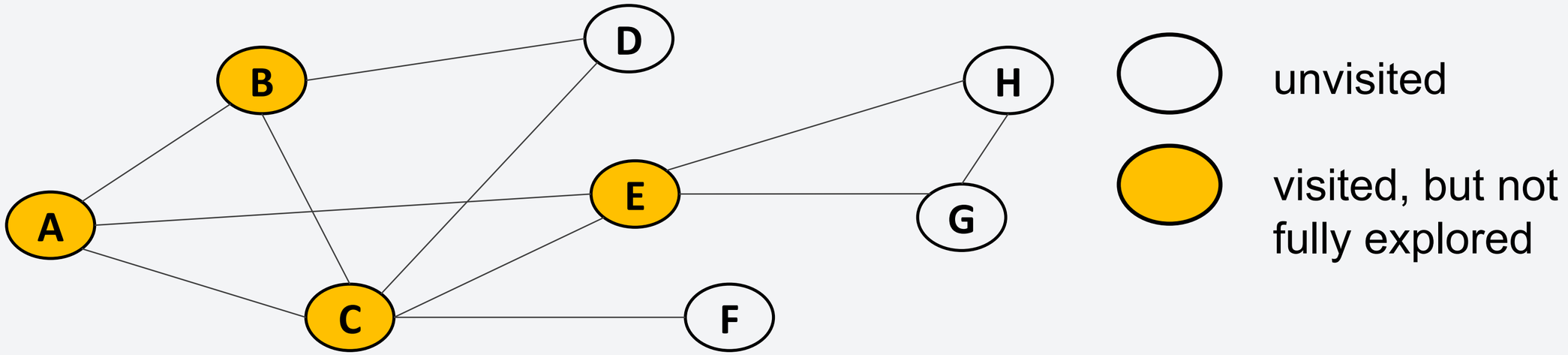
Queue: A

Visited: A

BFS Example

Given the following graph, let's do a BFS:

- Dequeue A.
- Visit the unvisited neighbors of A (B, C, E) and enqueue them.



Heuristic Function: Alphabetical order

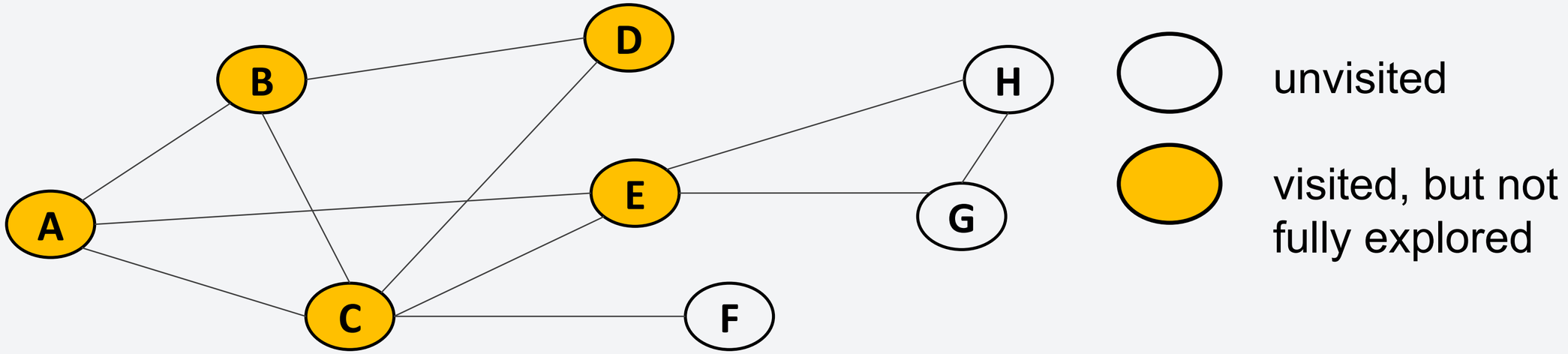
Queue: B, C, E

Visited: A, B, C, E

BFS Example

Given the following graph, let's do a BFS:

- Dequeue B.
- Visit the unvisited neighbors of B (D) and enqueue.



Heuristic Function: Alphabetical order

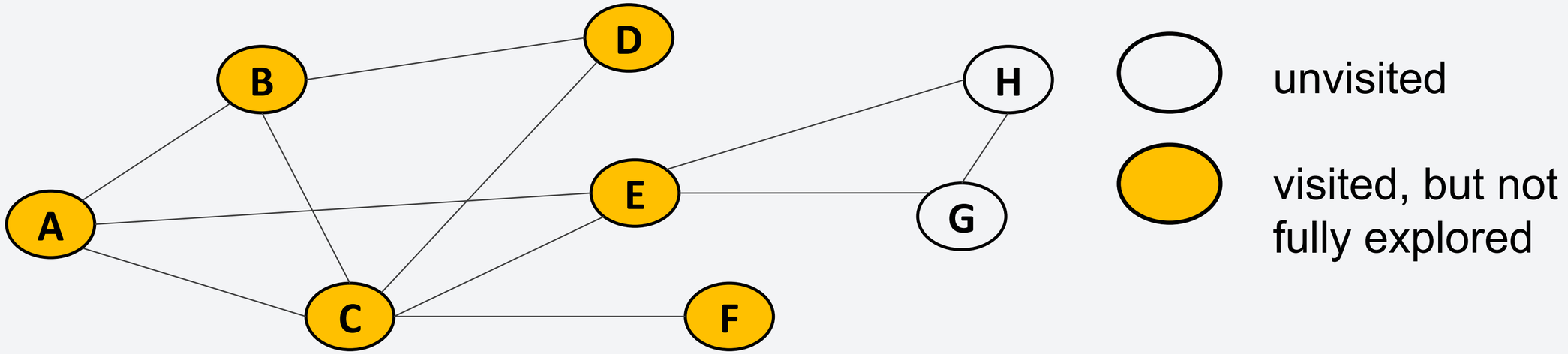
Queue: C, E, D

Visited: A, B, C, E, D

BFS Example

Given the following graph, let's do a BFS:

- Dequeue C.
- Visit the unvisited neighbors of C (F) and enqueue.



Heuristic Function: Alphabetical order

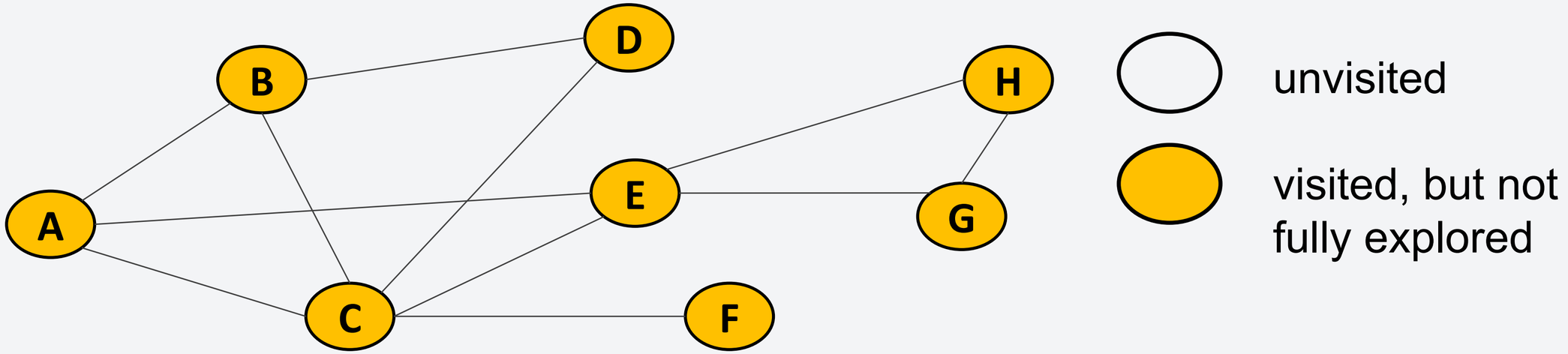
Queue: E, D, F

Visited: A, B, C, E, D, F

BFS Example

Given the following graph, let's do a BFS:

- Dequeue E.
- Visit the unvisited neighbors of E (G and H) and enqueue.



Heuristic Function: Alphabetical order

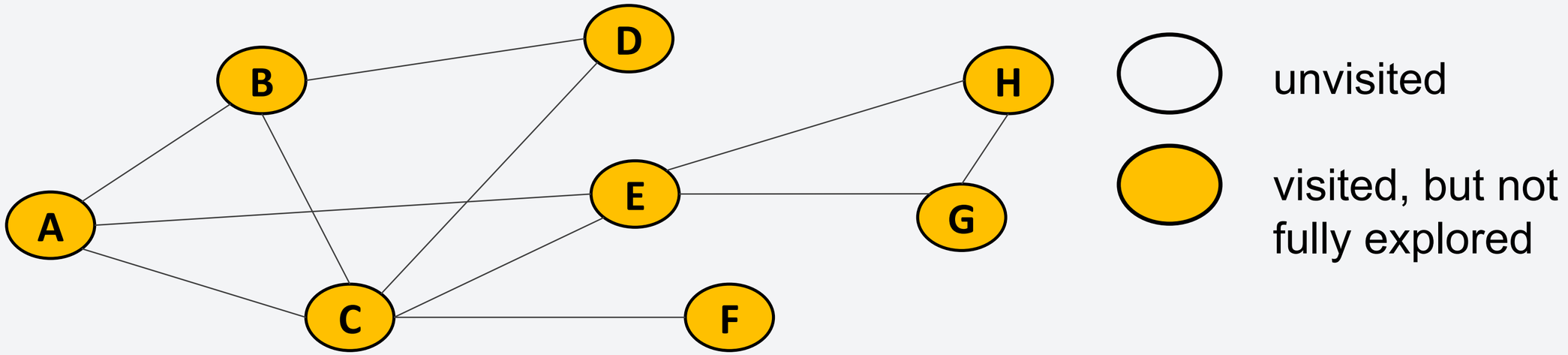
Queue: E, D, F, G, H

Visited: A, B, C, E, D, F, G, H

BFS Example

Given the following graph, let's do a BFS:

- All nodes have been visited. Just dequeue until the queue is empty.



Heuristic Function: Alphabetical order

Queue:

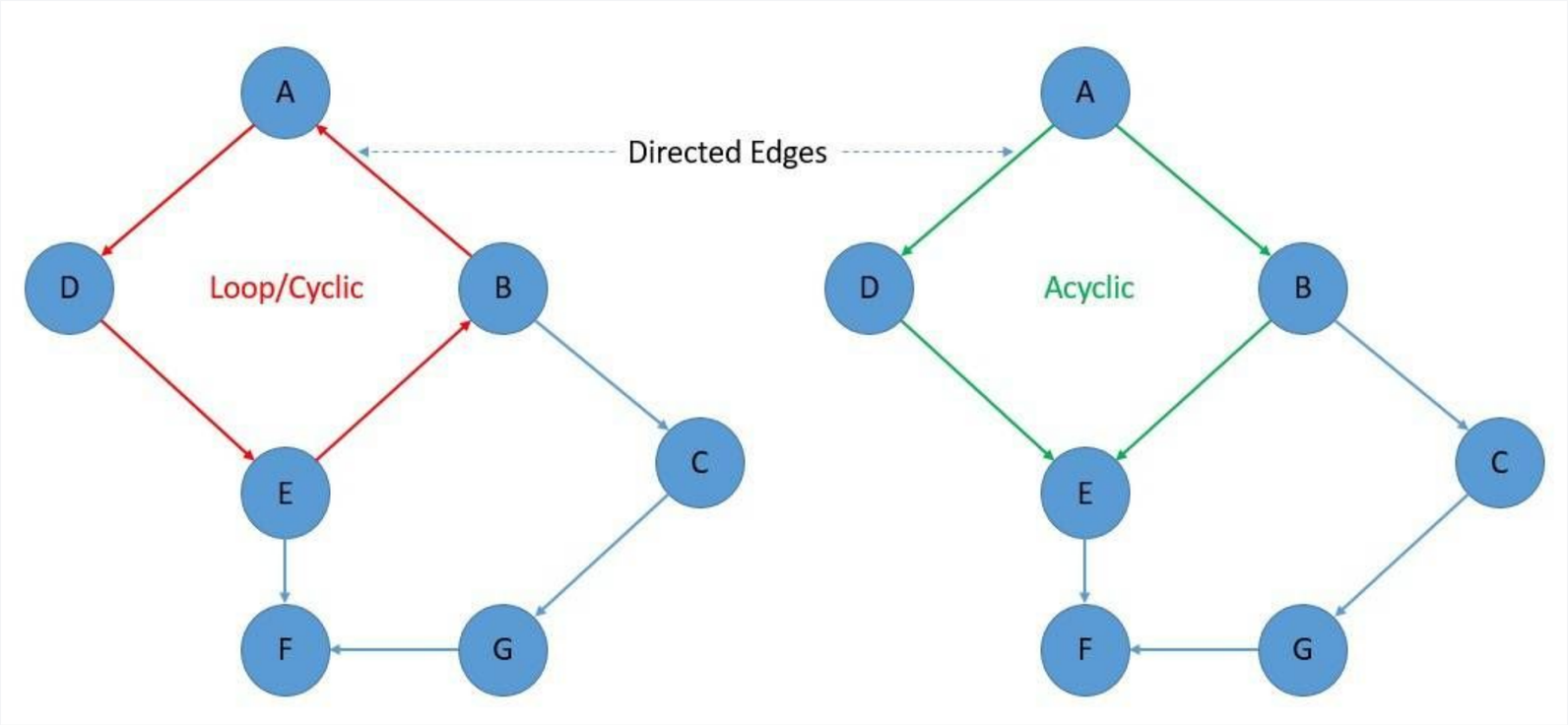
Visited: A, B, C, E, D, F, G, H

Demo: Breadth-first Search

- Graph Representation
- Graph Traversal
- **Directed Acyclic Graphs**

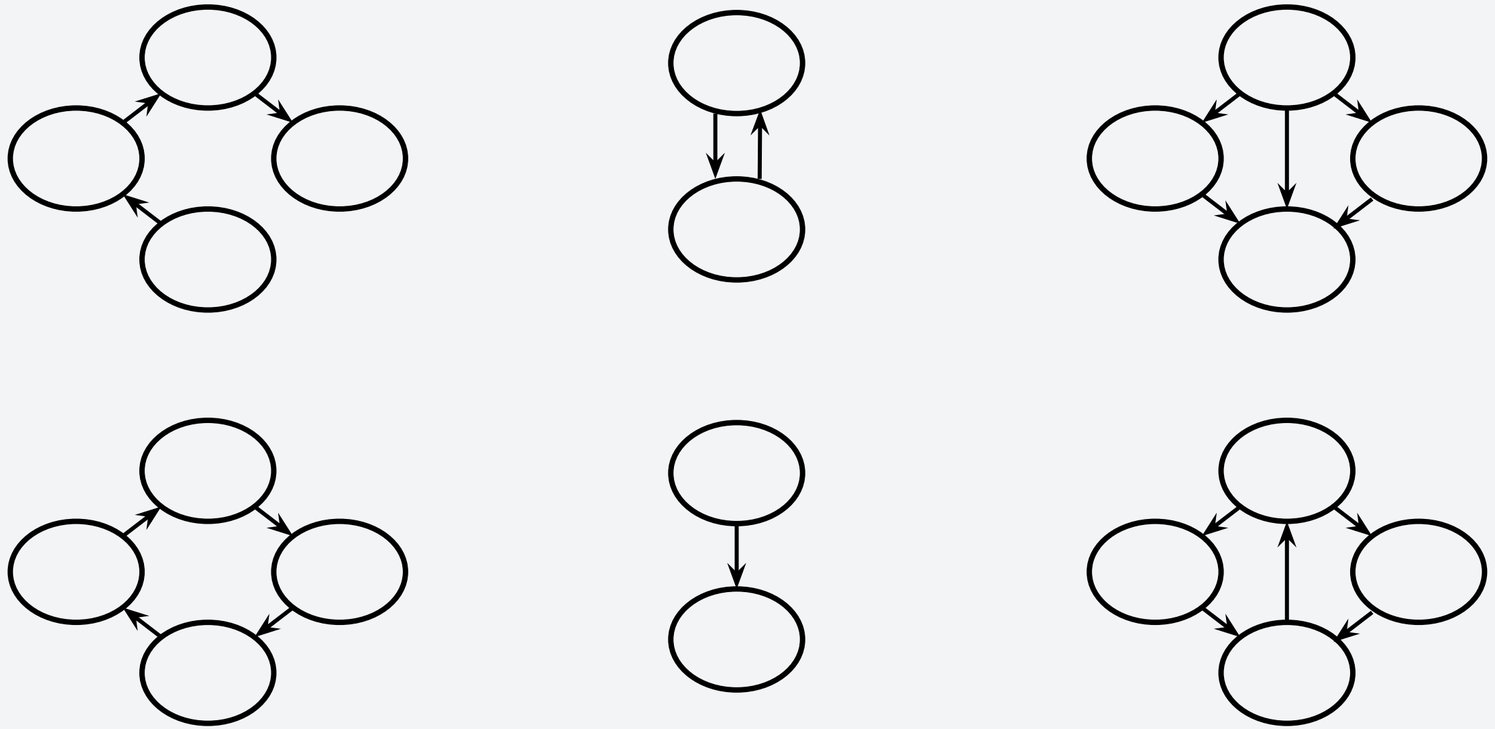
Directed Acyclic Graphs

An acyclic graph is a graph where nodes are linked by one-way edges and does not form a cycle or a loop. This means that there is no connection that will allow you to traverse the graph from a starting node and back to it.



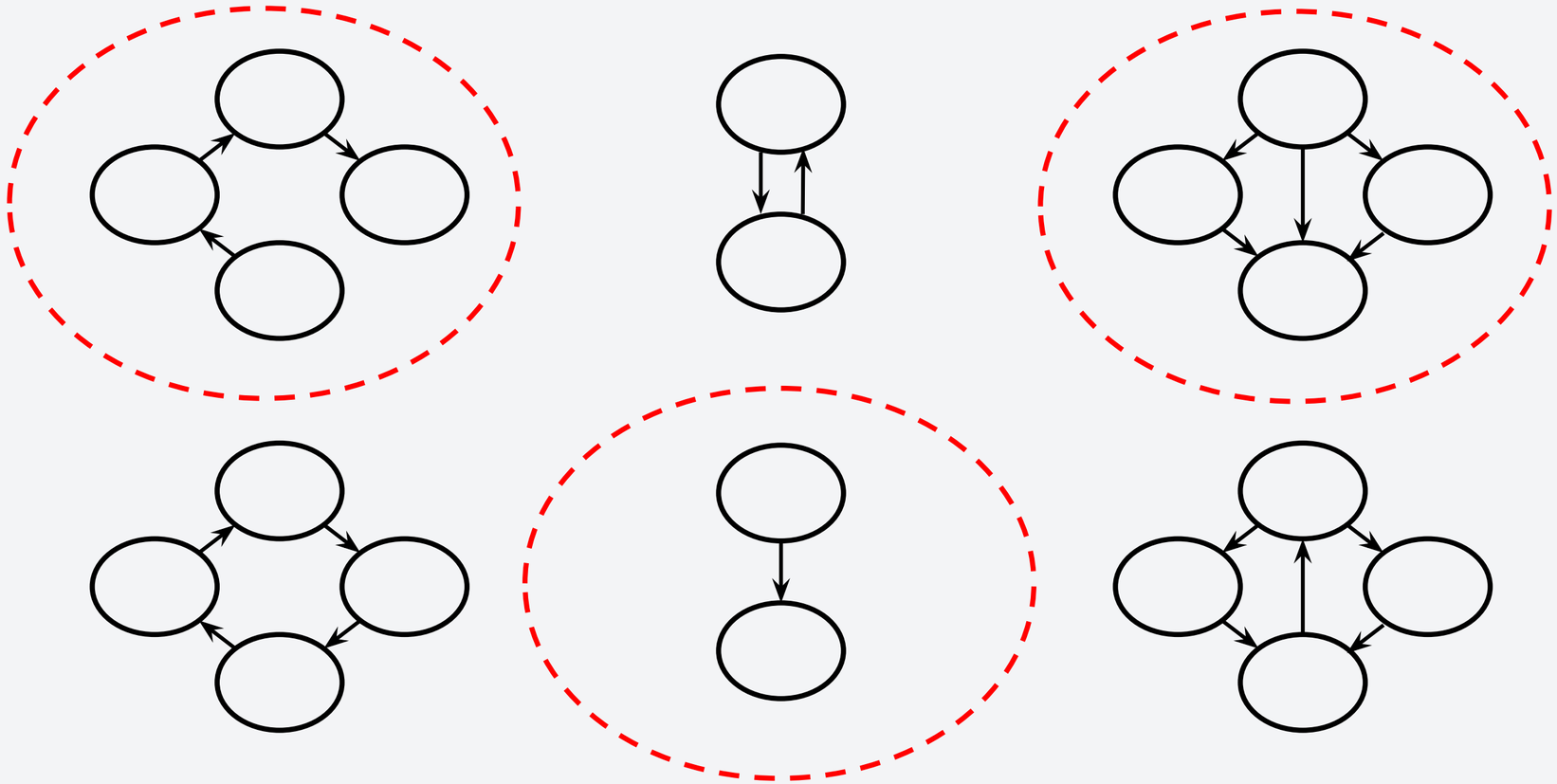
Directed Acyclic Graphs

Which of the following are directed acyclic graphs?



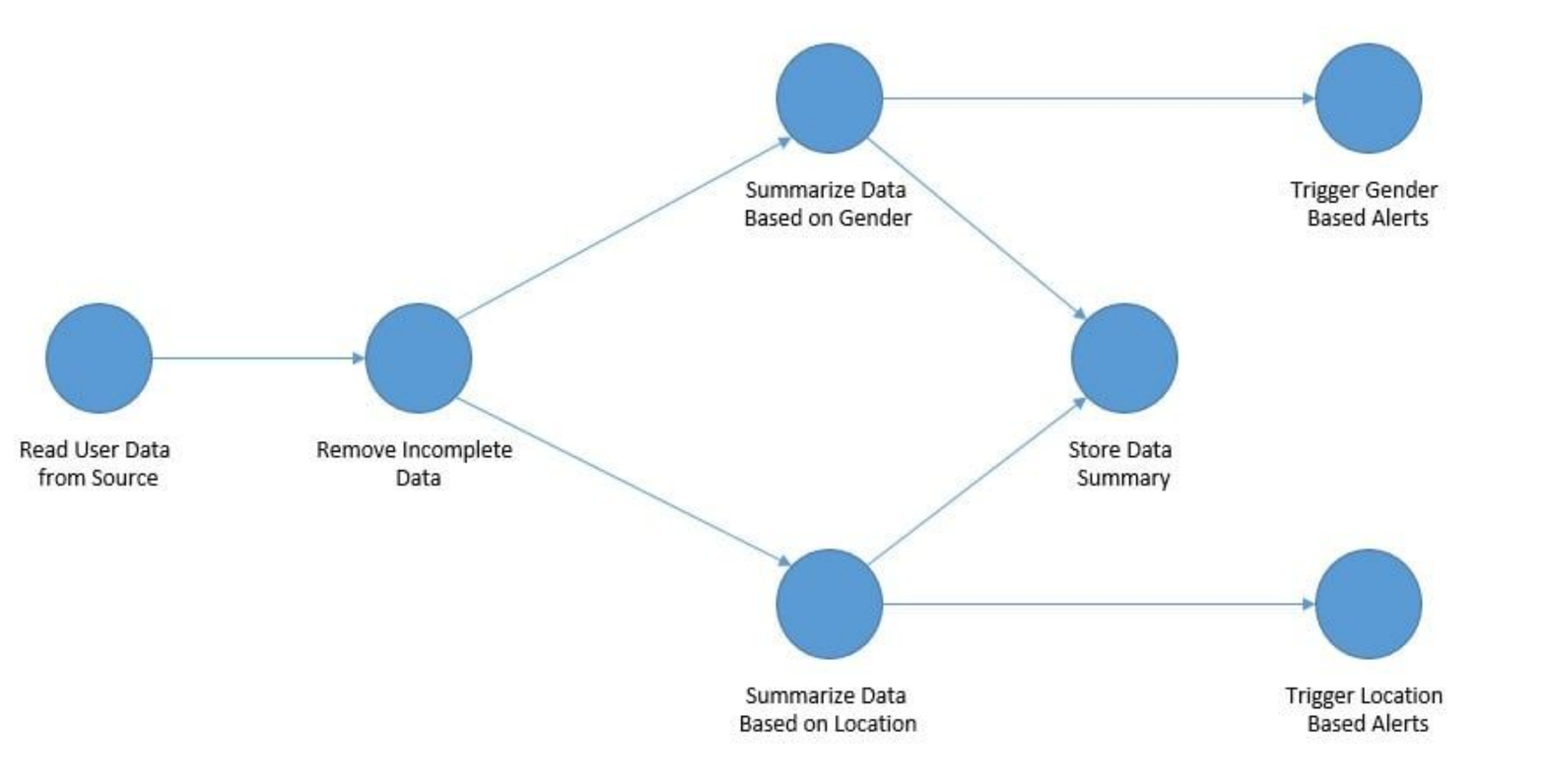
Directed Acyclic Graphs

Which of the following are directed acyclic graphs?



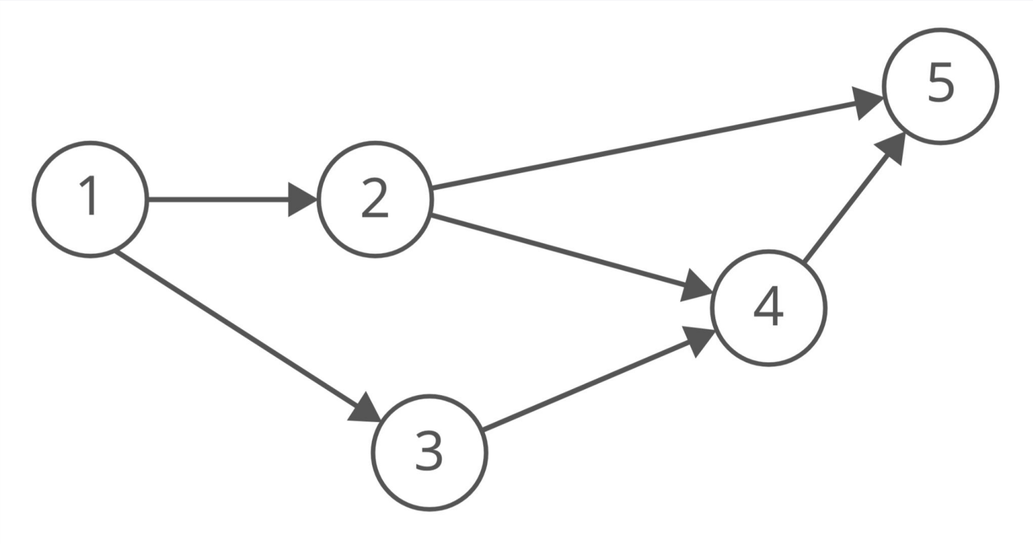
Why use DAGs?

Directed acyclic graphs are useful when modeling a series of tasks that a certain system has to perform. For example, DAGs are widely used in data processing systems,



Topological Ordering

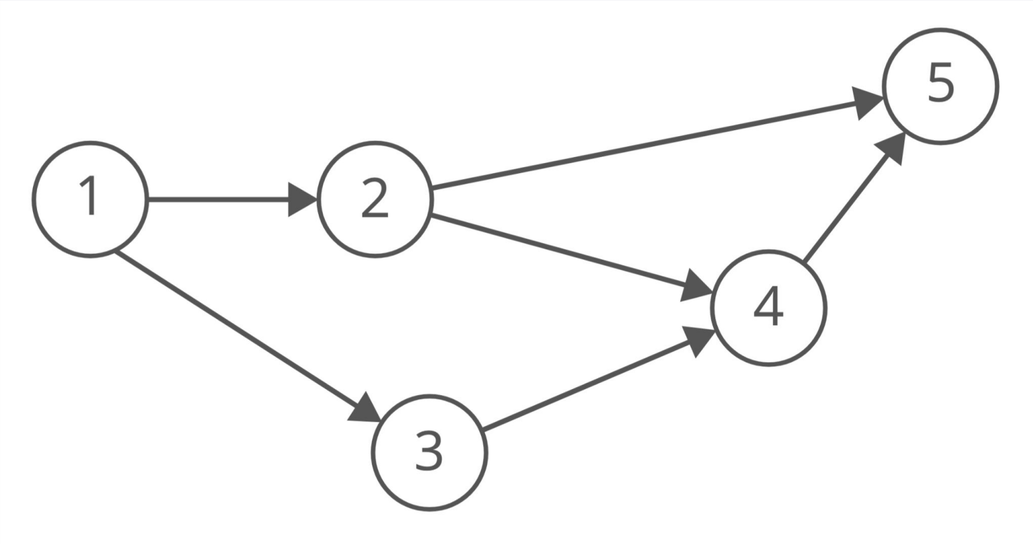
The topological ordering of a DAG is an ordering of its vertices such that for every directed edge (u,v) in the set of edges, u will always precede v.



Consider the graph on the right, can we find a topological ordering for its vertices?

Topological Ordering

The topological ordering of a DAG is an ordering of its vertices such that for every directed edge (u,v) in the set of edges, u will always precede v.

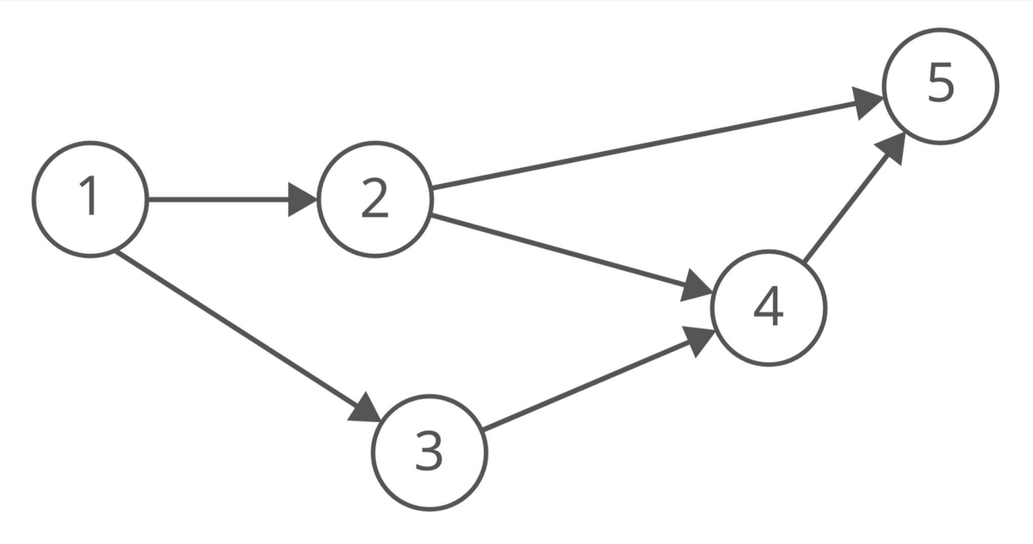


Consider the graph on the right, can we find a topological ordering for its vertices?

- Node 1 points to Node 2 and 3. Node 1 will always precede Nodes 2 and 3.
- Node 2 and 3 points to Node 4. Nodes 2 and 3 will always precede Node 4.
- Node 2 and 4 points to Node 5. Nodes 2 and 4 will always precede Node 5.

Topological Ordering

The topological ordering of a DAG is an ordering of its vertices such that for every directed edge (u,v) in the set of edges, u will always precede v.



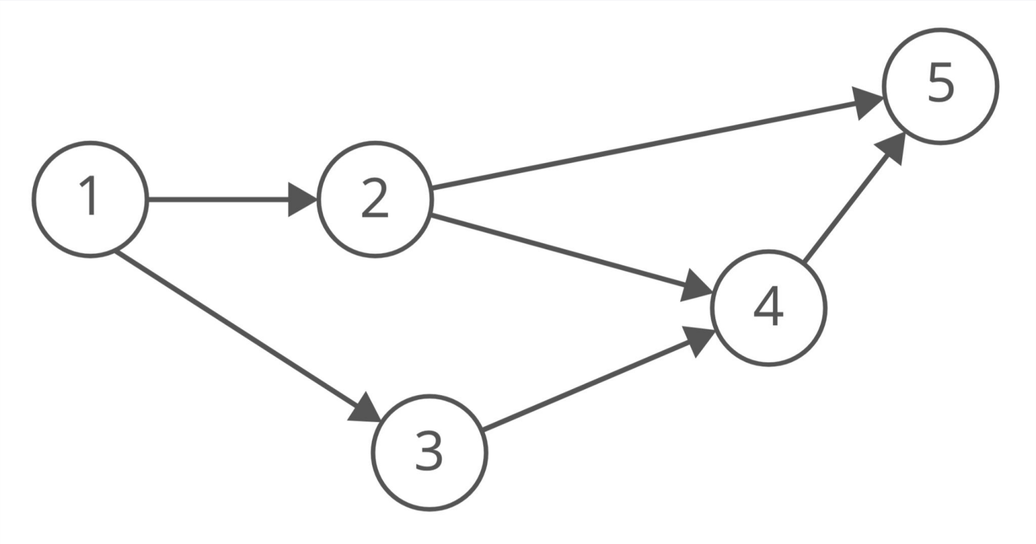
[1, 2, 3, 4, 5]

Consider the graph on the right, can we find a topological ordering for its vertices?

- Node 1 points to Node 2 and 3. Node 1 will always precede Nodes 2 and 3.
- Node 2 and 3 points to Node 4. Nodes 2 and 3 will always precede Node 4.
- Node 2 and 4 points to Node 5. Nodes 2 and 4 will always precede Node 5.

Topological Ordering

The topological ordering of a DAG is an ordering of its vertices such that for every directed edge (u,v) in the set of edges, u will always precede v.



[1, 2, 3, 4, 5] or [1, 3, 2, 4, 5]

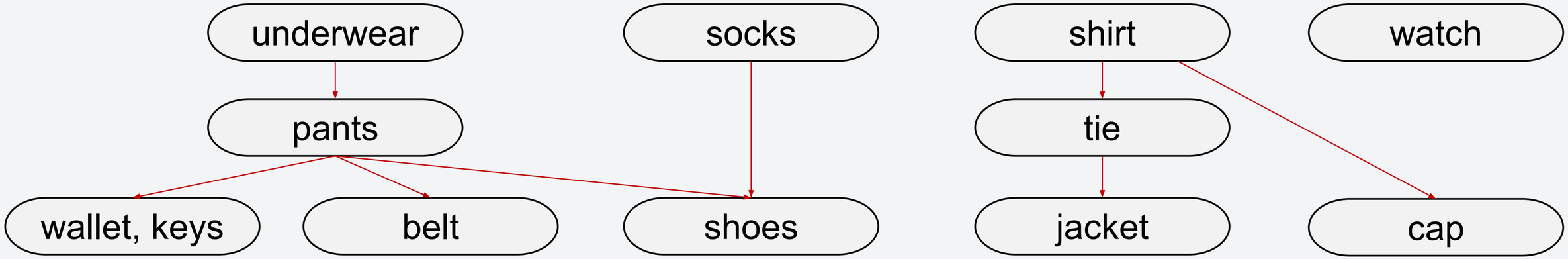
NOT UNIQUE!

Consider the graph on the right, can we find a topological ordering for its vertices?

- Node 1 points to Node 2 and 3. Node 1 will always precede Nodes 2 and 3.
- Node 2 and 3 points to Node 4. Nodes 2 and 3 will always precede Node 4.
- Node 2 and 4 points to Node 5. Nodes 2 and 4 will always precede Node 5.

Topological Ordering - Another Example

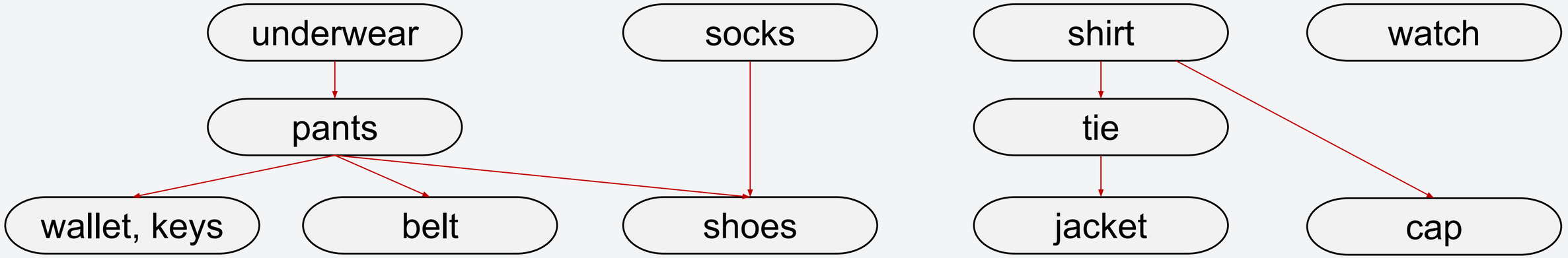
Consider the following graph:



Can you determine a possible topological ordering for the graph?

Topological Ordering - Another Example

Consider the following graph:

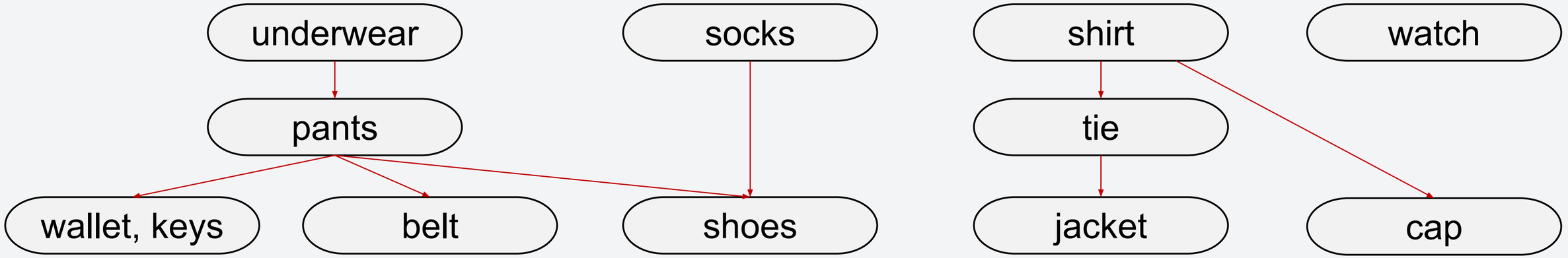


Can you determine a possible topological ordering for the graph?

Underwear, socks, pants, wallet, keys, belt, shoes, shirt, tie, jacket, cap, watch

Topological Ordering - Another Example

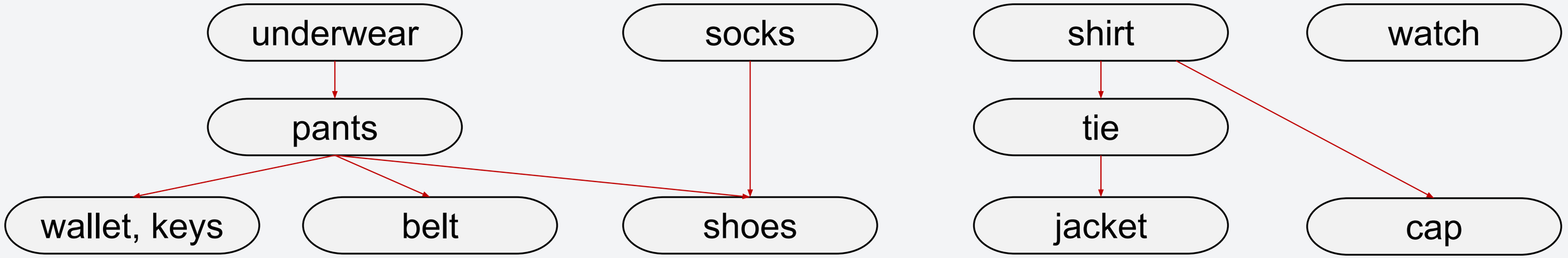
Consider the following graph:



Is belt, underwear, pants, ... valid?

Topological Ordering - Another Example

Consider the following graph:



Is belt, underwear, pants, ... valid?

NO! Why would you wear belt before pants? Underwear before pants?

Superman yarn?

Demo: Topo Sort

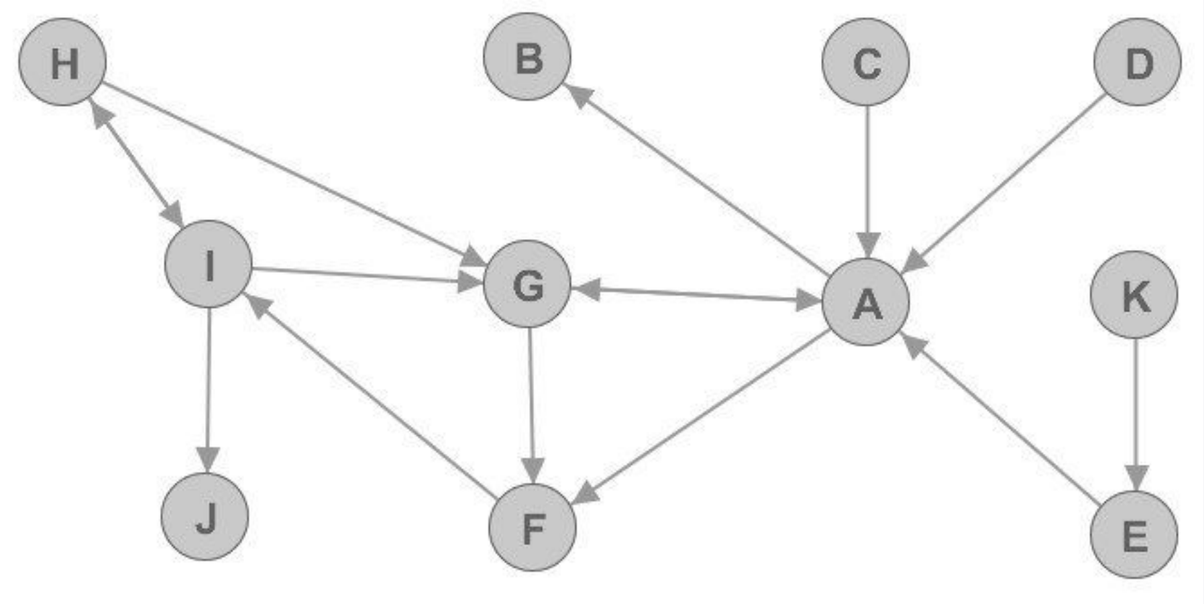
Any questions?

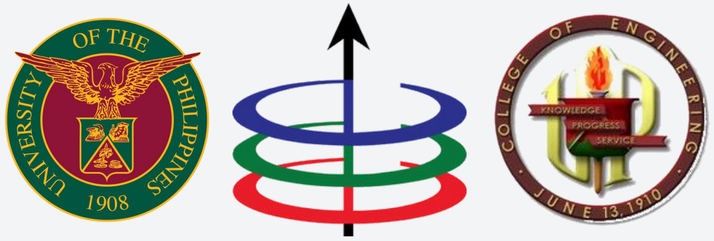
Quiz tayo! FR this time :D

Quiz!

Consider the following directed graph:

1. What are the neighbors of A?
2. What is the indegree of G? outdegree?
3. Construct the adjacency matrix and adjacency list representation of the graph.
4. Perform DFS, provide the order of the explored nodes (choose the letter that comes first in the alphabet).
5. Perform BFS, provide the order of the visited nodes (use the same heuristic as item 4).





Lecture 6
Graph Representation and Traversal

EEE 121 2s2526

Steven S. Sison